



AWS Solutions Architect Associate Cheat Sheets

These cheat sheets are provided for personal and non-commercial use only.

Please do not redistribute or upload these cheatsheets elsewhere.

Good luck on your exam!



S3 CheatSheet

Amazon S3 Fundamentals

- Amazon S3 is a **fully managed, regional object storage service** that stores data as **objects** inside **buckets**.
- Each **object** consists of:
 - **Key** – the full object name (including prefixes)
 - **Value** – the actual data (0 bytes to 5 TB)
 - **Version ID** – present when versioning is enabled
 - **Metadata** – system-defined or user-defined key–value pairs
- Buckets are **globally unique** across an AWS partition
- Objects are stored in a **flat namespace**
- “Folders” are an illusion created by prefixes
- Maximum object size is **5 TB**
- Multipart upload is **required for objects > 5 GB**

S3 Bucket Configuration

- **Naming Rules:** Unique names, 3-63 characters, only lowercase letters, numbers, dots, and hyphens.
- **Restrictions:** Default 100 buckets, can request up to 1000.
- **Versioning:** Keeps multiple versions of an object.
- **Encryption:** Server-side (SSE-S3, SSE-KMS) and client-side encryption.



S3 CheatSheet

Bucket Types

General Purpose Buckets (Default)

- Flat object namespace
- Works with **all storage classes except S3 Express One Zone**
- Supports versioning, lifecycle rules, replication, encryption
- Used for **most workloads**

Directory Buckets (Newer, Important)

- Required for **S3 Express One Zone**
- True directory-style hierarchy
- Designed for **ultra-low latency (single-digit ms)**
- **Single Availability Zone only**
- Limited number per account

Exam Tip: If you see single-digit millisecond latency + object storage, think S3 Express One Zone + Directory Bucket



S3 CheatSheet

S3 Object Concepts (Heavily Tested)

Object Prefixes

- Part of the object key
- Simulate folders using **'/'**
- No actual directories exist
- Max key length: **1024 bytes (prefix + filename)**

Etags

- Used to detect **content changes**
- Usually MD5, **but not guaranteed** (especially with encryption or multipart uploads)
- Represents a **specific object version**
- Does **not reflect metadata changes**

Checksums

- Used to validate **data integrity**
- Supported algorithms: CRC32, CRC32C, SHA-1, SHA-256
- Can be specified during upload

Exam Tip: Use **checksums** for integrity verification, **ETags** for change detection.



S3 CheatSheet

Object Metadata vs Tags (Classic Confusion)

Metadata

- Describes the object's contents
- Stored **with the object**
- Retrieved with the object
- Can be system-defined or user-defined (`x-amz-meta-*`)
- Cannot be modified without re-uploading the object

Object Tags

- Used for **cost allocation, access control, lifecycle rules**
- Stored separately from the object
- Can be modified without re-upload
- Limited to **10 tags per object**

Exam Tip: Lifecycle rules and IAM conditions work on tags, not metadata.



S3 CheatSheet

Object Lock (WORM) — Compliance Favourite

What Object Lock Does

- Enforces **Write Once, Read Many (WORM)**
- Prevents deletion or overwrite of objects
- Must be enabled at **bucket creation**
- Requires versioning

Retention Modes

- **Governance Mode**
 - Privileged users can override
 - Used for testing and flexibility
- **Compliance Mode**
 - **No one**, including root, can delete or shorten retention
 - Requires deleting the AWS account to remove data
- **Legal Holds**
 - No expiration date
 - Overrides retention periods
 - Remains until explicitly removed

Exam Tip: Regulatory keywords (SEC, FINRA, audit) → **S3 Object Lock**



S3 CheatSheet

S3 Storage Classes (Very High Exam Weight)

Frequently Accessed

- **S3 Standard:** Multi-AZ, 99.99% availability, no retrieval fee, no minimum duration

Infrequent Access

- **S3 Standard-IA:** Multi-AZ, retrieval fee, 30-day minimum
- **S3 One Zone-IA:** Single AZ, cheaper than Standard-IA, data loss risk

Intelligent

- **S3 Intelligent-Tiering:** Automatically moves objects, monitoring fee applies, multiple access tiers including archive

Ultra-Low Latency

- **S3 Express One Zone:** Single AZ, requires Directory Buckets, lowest latency object storage

Glacier Classes (Not a Separate Service Anymore)

- **S3 Glacier Instant Retrieval:** Millisecond access, 90-day minimum
- **S3 Glacier Flexible Retrieval:** Minutes to hours, Expedited, Standard, Bulk tiers
- **S3 Glacier Deep Archive:** 12–48 hour retrieval, lowest cost, 180-day minimum

Exam Tip: S3 Glacier Vaults are legacy. The exam focuses on S3 Glacier storage classes, not vaults.



S3 CheatSheet

Encryption (Extremely Common Exam Topic)

Encryption In Transit: Uses **TLS 1.2 or TLS 1.3**; SSL and TLS 1.0/1.1 are deprecated

Server-Side Encryption (Default ON)

- **SSE-S3:** AWS-managed keys, AES-256, no extra cost
- **SSE-KMS:** AWS KMS-managed keys, Audit logging, fine-grained access, Additional KMS costs,
 - Requires `kms:Decrypt` and `kms:GenerateDataKey`
- **SSE-C:** Customer-provided keys, AWS does not store the key, must supply key for every request

Bucket Keys: Reduce KMS request costs up to **99%**, cache a bucket-level data key

Client-Side Encryption: You encrypt data before upload, AWS never sees plaintext

Exam Tip: If audit, compliance, or cross-account access is mentioned → **SSE-KMS**



S3 CheatSheet

Versioning (Recovery & Protection)

Key rules:

- Disabled by default
- Once enabled, **cannot be disabled**
- Delete creates a **delete marker**
- Previous versions remain

Delete markers:

- Appear as current version
- Cause 404 on GET
- Must delete specific version ID to permanently remove

MFA Delete

- Requires root user
- CLI/API only
- Protects against accidental deletes

Exam Tip: Versioning + lifecycle rules = long-term data protection strategy



S3 CheatSheet

Lifecycle Rules (Cost Optimization)

Lifecycle rules can:

- Transition objects between storage classes
- Expire current or noncurrent versions
- Delete incomplete multipart uploads
- Rules apply to existing objects and new objects

Important constraints:

- You can move down, never back up
- Minimum storage durations still apply
- Objects < 128 KB often cannot be transitioned

Exam Tip: Lifecycle $\neq$ backup. It's cost management automation.



S3 CheatSheet

Replication (Disaster Recovery & Compliance)

Types

- **CRR** – Cross-Region Replication
- **SRR** – Same-Region Replication
- **Bi-Directional Replication**
- **Batch Replication** (existing objects)

Key requirements:

- Versioning must be enabled on both buckets
- Replication applies only to new objects
- Delete marker behavior is configurable

Replication Time Control (RTC):

- 99.99% within 15 minutes
- Additional cost

Exam Tip: “Existing objects need replication” → **S3 Batch Replication**



S3 CheatSheet

Access & Networking (Security Scenarios)

Block Public Access: Enabled by default, overrides bucket policies unless disabled, #1 S3 security protection

Bucket Policies vs ACLs: ACLs are **legacy**. Bucket policies are preferred. ACLs cannot deny or use conditions

Access Points: Named endpoints with separate policies. Can restrict access by VPC or Internet. Simplify large-scale access control

Multi-Region Access Points: Use **AWS Global Accelerator**. Route to lowest-latency bucket

Object Lambda: Modify object output **without changing original**. Uses Lambda on GET, LIST, HEAD

Exam Tip: Transform data on read → **S3 Object Lambda**

API CheatSheet

- An **AWS API** is an **HTTPS-based interface** that allows applications, SDKs, and tools to interact with AWS services programmatically.
- Every AWS service exposes a **service endpoint** (a URL) that receives requests such as `CreateBucket`, `RunInstances`, or `PutObject`.
- API requests must include:
 - **An action (operation)** (for example: `ec2:RunInstances`)
 - **Parameters** (instance type, AMI ID, bucket name, etc.)
 - **Authentication and authorization** (via signed requests)

Exam Tip: You are not expected to manually craft API requests, but you **must understand that the Console, CLI, SDKs, and Infrastructure-as-Code tools all call the same AWS APIs underneath.**

How Users Interact with AWS APIs (Most → Least Common)

- **AWS Management Console** – Web UI (still API-driven)
- **AWS SDKs** – Language-specific libraries (Python, Java, JavaScript, Go, etc.)
- **AWS CLI** – Command-line tool
- **Direct HTTPS requests** – Rare, advanced use cases

Exam Tip: If a question asks “what does the AWS CLI do?”, the correct conceptual answer is:

- **It makes signed API calls to AWS service endpoints on your behalf.**

API CheatSheet

Authentication & Credentials (Very High Exam Weight)

Access Keys

- An access key consists of `AccessKeyId` and `SecretAccessKey`.
- Used for **programmatic access only** (CLI, SDKs, APIs).
- Access keys **inherit all permissions of the IAM identity** they belong to.

Critical rules (exam loves these):

- Never share access keys.
- Never hardcode them in source code.
- IAM users can have **up to two active access keys** (for rotation).
- Keys can **be disabled or deleted** without deleting the user.

Credential Storage & Resolution Order

- Stored locally in `~/.aws/credentials` (credentials) and `~/.aws/config` (regions, output format, roles, SSO, retry mode)
- AWS tools automatically resolve credentials in this priority order:
 1. **CLI flags**
 2. **Environment variables**
 3. **Credentials/config files**
 4. **Instance metadata / container credentials**

Exam Tip: **Environment variables override config files**, but are overridden by explicit CLI parameters.

API CheatSheet

AWS STS – Temporary Credentials (Extremely Important)

AWS Security Token Service (STS) issues **temporary, short-lived credentials**. Used for:

- **IAM role assumption**
- **Cross-account access**
- **Federated users (SSO, web identity, SAML)**
- **Enhanced security (no long-lived keys)**

STS returns:

- ``AccessKeyId``
- ``SecretAccessKey``
- ``SessionToken``
- ``Expiration``

Common STS API Actions:

- ``AssumeRole``
- ``AssumeRoleWithWebIdentity``
- ``AssumeRoleWithSAML``
- ``GetCallerIdentity``
- ``GetSessionToken``

Exam Tip: If a question mentions **temporary credentials, role assumption, or cross-account access**, the answer almost always involves **STS**.

API CheatSheet

API Request Signing (Signature Version 4)

AWS Signature Version 4 (SigV4)

- **All modern AWS services use SigV4.**
- Ensures **Request integrity** (no tampering) and **Authentication** (who sent the request)
- Uses:
 - Access key
 - Secret key
 - Region
 - Service name
 - Date

Important clarification (often misunderstood):

- The **secret access key is used to create a signing key, not sent directly.**

Exam Tip: You don't need to memorize the math — just know **CLI and SDKs sign requests automatically.**

API CheatSheet

Service Endpoints (Medium–High Exam Weight)

Endpoint Types

1. Regional endpoints

- Require a region (e.g., `ec2.us-east-1.amazonaws.com`)
- Used by most services (EC2, S3, Lambda, RDS, ECS, etc.)

2. Global endpoints

- Single endpoint for all regions
- Examples:
 - **IAM**
 - **Route 53**
 - **CloudFront**
 - **Global Accelerator**

3. FIPS endpoints

- For government / regulated workloads
- Use FIPS-validated cryptography

4. Dualstack endpoints

- Support **IPv4 and IPv6**
- Common with S3 and some EC2 APIs

Exam Tip: If a service manages **global resources**, it likely uses a **global endpoint**.



CLI CheatSheet

AWS CLI Essentials

- A **command-line tool** that sends **signed API requests** to AWS services.
- Written in Python and powered by **botocore**.
- Current version: **AWS CLI v2** (v1 is legacy and should not be used).

CLI Command Structure

aws <service> <operation> [parameters]

Example: **aws ec2 run-instances --image-id ami-123 --instance-type t3.micro**

- **Service name** → maps to an AWS service endpoint
- **Operation** → maps directly to an API action
- **Parameters** → map to request payload fields

Exam Tip: CLI commands, SDK calls, and IAM policy actions all use the same API action names.



CLI CheatSheet

IAM Policies & API Actions

IAM permissions reference **API actions**, not CLI commands. Example: `"Action": "ec2:RunInstances"`

This same action is used by:

- AWS CLI
- AWS SDKs
- CloudFormation
- Terraform
- Console

Exam Tip: If permission is missing in IAM, **no tool can bypass it.**

Retries & Exponential Backoff - Why Retries Exist

- Network failures are expected.
- AWS SDKs and CLI implement automatic retries using **exponential backoff**.

Retry Modes (CLI v2)

- **Standard (default)** – Balanced retry behavior
- **Adaptive** – Adjusts retry rate dynamically
- **Legacy** – Older behavior (not recommended)

Exam Tip: If a question mentions **throttling**, **API limits**, or **retry logic**, exponential backoff is the correct concept. ¹⁹



CLI CheatSheet

IAM Roles vs Access Keys (Very Common Exam Question)

Why IAM Roles Are Preferred

- No long-lived credentials
- Automatically rotated
- Use STS temporary credentials
- Required for: EC2 instance permissions, ECS task roles, and Cross-account access

Exam Tip: If AWS is calling AWS, use IAM roles — not access keys.

Environment Variables (High Exam Relevance)

Common ones:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`
- `AWS_SESSION_TOKEN`
- `AWS_REGION`
- `AWS_PROFILE`

Important: Environment variables **override config files** but are overridden by CLI parameters.

Amazon Virtual Private Cloud (VPC) CheatSheet

VPC Core Concept

- **Amazon VPC** is a **logically isolated virtual network** inside an AWS Region where you define IP ranges, subnets, routing, and security.
- Conceptually similar to an **on-premises data center network**, but fully software-defined and managed by AWS.
- A VPC is **regional** (does not span Regions).
- Almost all VPC-related CLI/API operations live under the **EC2 API namespace** (e.g., `aws ec2 create-vpc`), even when the resource is not an EC2 instance.

VPC Core Components (Must Know for Exam)

Subnets: A **subnet** is a range of IP addresses within a VPC.

- Subnets are **Availability Zone–specific**.
- A subnet is considered **public** if its route table has a route to an **Internet Gateway (IGW)**.
- A subnet is considered **private** if it does **not** have a route to an IGW.

Route Tables

- Route tables determine **where network traffic is directed**.
- Every subnet must be associated with **exactly one route table** (explicit or implicit).
- Each VPC has a **Main Route Table** (cannot be deleted).
- Custom route tables are commonly used to separate **public and private subnet behaviour**.

Exam Tip: If a subnet is not explicitly associated with a route table, it uses the **Main Route Table**.

Amazon Virtual Private Cloud (VPC) CheatSheet

Internet Gateway (IGW)

- Enables **inbound and outbound internet access** for IPv4 and IPv6.
- Must be:
 1. Created
 2. Attached to a VPC
 3. Referenced in a route table (`0.0.0.0/0` or `::/0`)
- Public IPv4 traffic does not require NAT if the instance has a public IP.

Egress-Only Internet Gateway (IPv6)

- Used **only for IPv6**.
- Allows **outbound-only** internet access.
- Prevents unsolicited inbound IPv6 traffic.
- Required because IPv6 addresses are **globally routable by default**.



NACLs and Security Groups CheatSheet

Security Layers: SGs vs NACLs (High-Frequency Exam Topic)

Security Groups (SGs)

- **Stateful** virtual firewalls.
- Operate at the **instance (ENI) level**.
- **Allow rules only** (implicit deny).
- If inbound traffic is allowed, **return traffic is automatically allowed**.
- Can reference: IPv4 CIDR, IPv6 CIDR, **Other security groups**, and Managed prefix lists
- Instances can belong to **multiple SGs** (rules are permissive).

Exam Tip: You **cannot block a specific IP** using a Security Group.

Network ACLs (NACLs)

- **Stateless** firewalls.
- Operate at the **subnet level**.
- Support **explicit ALLOW and DENY** rules.
- Inbound and outbound rules are evaluated **independently**.
- Rules are processed **in order (lowest number first)**.
- Each subnet can be associated with **only one NACL**.

Exam Tip: Use **NACLs** when you must **explicitly deny traffic**, such as blocking a malicious IP.

Amazon Virtual Private Cloud (VPC) CheatSheet

Public vs Private Subnets (Exam Scenarios)

Requirement	Solution
Public internet access	IGW + route 0.0.0.0/0
Private outbound IPv4	NAT Gateway
Private outbound IPv6	Egress-Only IGW
No internet at all	No IGW, no NAT

NAT Gateway (Preferred, Exam-Relevant)

- Fully managed service for **outbound IPv4 access** from private subnets.
- Deployed in a **public subnet**.
- Requires an **Elastic IP**.
- **AZ-scoped** → create one NAT Gateway **per AZ** for high availability.
- Automatically scales up to **45 Gbps**.
- **Pricing Reminder (Exam-Relevant):** Charged **per hour** and **per GB processed**; Can become expensive at scale.

NAT Instance (Legacy – Know Why It's Bad)

- EC2-based NAT solution.
- Requires: Source/destination check disabled; Manual scaling; Patching and maintenance
- **Not recommended** for modern architectures.
- Amazon Linux NAT AMIs are **end-of-life**.

Exam Tip: If asked “best” or “recommended” → **NAT Gateway**, not NAT Instance.

Amazon Virtual Private Cloud (VPC) CheatSheet

Elastic IPs (EIP)

- Static **public IPv4 addresses**.
- Region-scoped.
- Can be associated with EC2 instances, ENIs, or NAT Gateways
- **Charged when allocated but not associated.**
- IPv6 does **not** require Elastic IPs.

VPC Peering

- Direct network connection between two VPCs using **private IPs**.
- Works across Accounts and Regions
- **No transitive routing.**
- **CIDR blocks must not overlap.**
- Each VPC must have route table updates pointing to the peering connection.

Exam Tip: If transitive routing is required → use **Transit Gateway**, not peering.

Transit Gateway (Hub-and-Spoke)

- Central routing hub for: Multiple VPCs, On-premises networks, and VPN and Direct Connect
- Supports up to **5,000 VPC attachments**.
- Enables **transitive routing**.
- Integrates with **AWS RAM** for cross-account sharing.



VPC Endpoint and VPC Flow Logs CheatSheet

VPC Endpoints & Private Connectivity

Gateway Endpoints

- Free.
- Route-table based.
- Supported services: **Amazon S3** and **Amazon DynamoDB**
- Do **not** use PrivateLink.

Interface Endpoints (AWS PrivateLink)

- ENI-based private endpoints.
- Support **many AWS services** (SQS, SNS, KMS, STS, CloudWatch, API Gateway, etc.).
- Charged **per AZ per hour** + data processing.
- Keep traffic **inside AWS**, no IGW or NAT required.

Gateway Load Balancer Endpoints

- Used for **third-party network appliances** (firewalls, IDS/IPS).
- Integrates with **Gateway Load Balancer**.
- Traffic routing controlled via route tables.



VPC Endpoint and VPC Flow Logs CheatSheet

VPC Flow Logs

- Capture IP traffic metadata (not payload).
- Can be enabled at: VPC, Subnet, ENI, and Transit Gateway
- Deliver logs to: CloudWatch Logs, Amazon S3, and Kinesis Data Firehose
- Cannot be modified after creation.

Important Exam Caveat

- Flow Logs **do not capture**: DNS traffic, DHCP traffic, Instance metadata traffic, or Windows license activation

Amazon Virtual Private Cloud (VPC) CheatSheet

VPN Connectivity

Site-to-Site VPN

- IPSec tunnels between on-prem and AWS.
- Uses: Customer Gateway (CGW), Virtual Private Gateway (VGW) **or** Transit Gateway
- Two tunnels per connection for HA.
- IPv6 supported **only via Transit Gateway**, not VGW.

Client VPN

- Managed client-based VPN.
- Supports Certificate auth, Active Directory, and SAML federation
- Uses **Security Groups** for access control.
- Ideal for **remote user access** to private resources.

AWS Direct Connect

- Dedicated private connection between on-prem and AWS.
- More **consistent and predictable** than internet VPN.
- Charged by Port hours and Data transfer out
- Can integrate with **Transit Gateway** for scale.

Amazon Virtual Private Cloud (VPC) CheatSheet

Advanced / Emerging (Know at High Level)

VPC Lattice

- Managed **service-to-service** networking.
- Supports: Cross-VPC, Cross-account, and IAM-based access
- Uses HTTP/S and supports canary/blue-green routing.
- Think of it as **application-level networking**, not traditional VPC routing.

Traffic Mirroring

- Mirrors ENI traffic to monitoring appliances.
- Used for **security inspection** and analysis.
- Uses VXLAN encapsulation.

AWS Network Firewall

- Managed **stateful firewall and IDS/IPS**.
- Uses Suricata-compatible rules.
- Protects traffic at the **VPC perimeter**.



AWS Lambda CheatSheet

- **AWS Lambda** is a **serverless Function-as-a-Service (FaaS)** that lets you run code without provisioning, managing, or patching servers.
- Lambda automatically **scales horizontally** by creating additional execution environments per request.
- You are billed **only when your code runs**, measured by **requests + execution duration × memory**.
- There is **no cost when the function is idle**.

Supported Runtimes & Architectures

Managed Runtimes (Most Common on Exam)

- **Node.js, Python, Java, C# (.NET), Go, Ruby, PowerShell**
- **Runtimes are fully managed and security-patched by AWS**
- Older runtimes are **deprecated** and require upgrades

Instruction Set Architectures

- **x86_64** – traditional Intel/AMD
- **arm64 (AWS Graviton)**
 - Lower cost and better price-performance
 - Preferred **when compatible**
- Architecture must match compiled binaries



AWS Lambda CheatSheet

Invocation Models (VERY IMPORTANT)

Synchronous Invocation

- Caller **waits for response**
- Errors returned directly to caller
- Common examples: **API Gateway** and **AWS SDK Invoke**
- Caller controls retries and timeouts

Asynchronous Invocation

- Caller **returns immediately**
- Lambda processes event in background
- AWS handles **retries automatically**
- Supports **Lambda Destinations**
- Common examples: **S3 events**, **SNS**, and **EventBridge**

Lambda Destinations (Async Only)

Used to route results of async executions. Destinations can send results to: **Amazon SNS**, **Amazon SQS**, **Amazon EventBridge**, **Another Lambda**

Triggers: **On success** and **On failure**

Exam Tip: Destinations $\neq$ DLQs (DLQs only capture failures)



AWS Lambda CheatSheet

Triggers & Event Sources

Lambda can be invoked by:

- **API Gateway**
- **Application Load Balancer**
- **S3**
- **DynamoDB Streams**
- **SQS**
- **SNS**
- **EventBridge**
- **CloudWatch Logs**
- **Cognito**
- **IoT**
- **Partner SaaS events via EventBridge**

Execution Limits & Configuration

Timeout - Default: 3 seconds; Max: 15 minutes

Memory - Min: 128 MB; Max: 10,240 MB (10 GB); Memory size also controls CPU & network throughput

Ephemeral Storage (/tmp) - Default: 512 MB; Max: 10 GB; Temporary — cleared when execution environment is destroyed



AWS Lambda CheatSheet

Persistent Storage Options

- **Amazon S3** (object storage)
- **Amazon EFS** (file system, mounted at `/mnt/efs``)
 - Requires VPC configuration
 - Persists data across invocations and compute services

Concurrency (High-Yield Exam Topic)

Default Concurrency: 1,000 concurrent executions per region per account

Reserved Concurrency

- Guarantees capacity for a function
- Also **limits max concurrency**
- No additional cost
- Prevents noisy neighbors

Provisioned Concurrency

- Keeps execution environments **pre-warmed**
- **Reduces or eliminates cold starts**
- **Billed even when idle**
- Must be attached to a **version** or an **alias**



AWS Lambda CheatSheet

Cold Starts

- Occur when Lambda creates a new execution environment
- More noticeable for: Java and VPC-attached Lambdas
- Mitigation strategies: Provisioned Concurrency and SnapStart (Java only)

Lambda SnapStart (Java Only)

- Improves cold start times by up to 10x
- Uses a Firecracker snapshot
- Supports Java 11+
- Does not support: arm64, Provisioned Concurrency, EFS (512 MB ephemeral storage)

Deployment Packages

ZIP Deployment

- Max direct upload: 50 MB
- Larger packages must be uploaded via S3
- Unzipped limit (code + layers): 250 MB
- Uses managed runtimes

Container Image Deployment

- Up to 10 GB
- Stored in Amazon ECR
- No runtime specified
- Full control over OS & dependencies



AWS Lambda CheatSheet

Lambda Layers

- Share libraries, SDKs, or runtimes
- Max **5 layers per function**
- Combined unzipped size counts toward **250 MB limit**
- Great for reuse and faster deployments

Versions & Aliases

Versions

- Immutable snapshots of code + config
- Each has a **unique ARN**
- ``$LATEST`` is mutable and not recommended for production

Aliases

- Human-friendly pointers to versions (e.g., ``prod``, ``dev``)
- Required for **Provisioned Concurrency** and Safe deployments

IAM Execution Role

- Grants Lambda permission to access AWS services
- Common managed policies: `AWSLambdaBasicExecutionRole`, `AWSLambdaVPCAccessExecutionRole`, `AWSLambdaDynamoDBExecutionRole`, `AWSLambdaSQSQueueExecutionRole`
- Lambda assumes this role automatically at runtime



AWS Lambda CheatSheet

Networking & VPC Access

- Lambdas **do not** run in your VPC by default
- To access VPC resources: Specify **subnets** + **security groups**
- Uses **Hyperplane ENIs**
- No extra charge for VPC networking
- Outbound internet access requires **NAT Gateway**

Environment Variables

- Key-value pairs available at runtime
- Encrypted at rest using **AWS KMS**
- Common predefined variables: ``AWS_REGION``, ``AWS_LAMBDA_FUNCTION_NAME``, ``AWS_LAMBDA_FUNCTION_MEMORY_SIZE``, ``_X_AMZN_TRACE_ID``

Logging & Monitoring

- Logs written automatically to **CloudWatch Logs**
- Supported logging methods:
 - ``console.log`` (Node.js)
 - ``logging`` (Python)
 - ``LambdaLogger`` (Java)
- JSON logs are preferred for analytics



AWS Lambda CheatSheet

Lambda Function URLs

- Built-in **HTTPS endpoint**
- Auth options: **NONE** (public) and **IAM** (SigV4)
- Supports **CORS**
- Simpler alternative to API Gateway (fewer features)

Security & Reliability Features

Code Signing

- Ensures only trusted code runs
- Uses: **AWS Signer** and **Lambda Code Signing Config**

Recursive Loop Detection

- Prevents infinite invocation loops
- Supports: SQS, SNS, Direct invokes
- Stops execution after **16 recursive invocations**
- Enabled by default



Amazon CloudFront CheatSheet

- **Amazon CloudFront** is a **global Content Delivery Network (CDN)** that delivers content to users with **low latency and high transfer speeds** by caching content at locations close to viewers.
- CloudFront is designed to handle:
 - **Static content** (HTML, CSS, JS, images)
 - **Dynamic content** (API responses, personalized pages)
 - **Streaming media** (video on demand and live streaming)
 - **WebSockets** (real-time bidirectional communication)
- CloudFront integrates tightly with:
 - **AWS WAF** for OWASP Top 10 protection
 - **AWS Shield** for DDoS mitigation

Exam Tip: CloudFront is often the **performance + security front door** for web architectures.

Core CloudFront Architecture Components

- The **origin** is the backend where CloudFront retrieves content from.
- Common origins: **Amazon S3 bucket, Application Load Balancer, Amazon EC2 (custom HTTP origin), AWS Elemental MediaStore, and Lambda Function URL**
- CloudFront **does not host content** itself, it caches content retrieved from the origin.



Amazon CloudFront CheatSheet

- **Edge locations** are globally distributed CloudFront points of presence.
- They cache content close to end users and serve cached responses directly when possible

Exam Tip: **Edge locations** $\neq$ **Regions** (they are separate from AWS Regions).

Regional Edge Caches

- Sit between edge locations and the origin.
- Improve cache hit ratios by:
 - Reducing origin fetches
 - Allowing less-frequently accessed content to stay cached longer
- Often tested implicitly in **performance optimization** scenarios.

Distribution

- A **CloudFront distribution** defines:
 - Which origin(s) to use
 - Cache behaviors
 - Security, caching, and routing rules
- Two types exist, but **only one matters for the exam**:
 - **Web distribution** (used for everything now)
 - RTMP distributions are deprecated and not tested



Amazon CloudFront CheatSheet

CloudFront Functions vs Lambda@Edge (VERY COMMON EXAM TOPIC)

CloudFront Functions

- Lightweight JavaScript functions that run at **edge locations**
- Best for **high-scale, ultra-low-latency** logic
- Supported events:
 - **Viewer Request**
 - **Viewer Response**
- Key characteristics:
 - JavaScript only (ECMAScript 5.1)
 - Sub-millisecond execution
 - No network, filesystem, or request body access
 - Very cheap and fast
- Common use cases:
 - Redirect HTTP → HTTPS
 - Add/remove headers
 - Cache key normalization
 - Simple authorization logic



Amazon CloudFront CheatSheet

Lambda@Edge

- Full Lambda functions deployed to **regional edge caches**
- Supports more complex logic than CloudFront Functions
- Supported events: Viewer Request, Viewer Response, Origin Request, and Origin Response
- Supports: **Node.js and Python**, Network access, and Request body access (with limits)
- Common use cases:
 - URL rewrites based on user agent
 - Authentication at the edge
 - Custom error responses
 - Dynamic origin selection
- Exam distinction:
 - **CloudFront Functions = speed + scale**
 - **Lambda@Edge = flexibility + power**

Origins and Origin Groups

Origin Groups

- Used for **high availability and failover**
- Consist of **Primary origin** and **Secondary (failover) origin**
- Failover occurs only for **GET, HEAD, OPTIONS** requests
- Failover triggers on specific HTTP status codes: 400, 403, 404, 416, 500, 502, 503, 504

Exam trap: CloudFront **does NOT fail over** for POST, PUT, PATCH, DELETE



CloudFront CheatSheet

Caching Fundamentals (Extremely Important)

Cache Key

- A **cache key uniquely identifies cached objects**
- If two requests generate the same cache key → **cache hit**
- Higher cache hit ratio = better performance and lower cost

Cache Policies

- Control **what goes into the cache key**
- AWS-managed cache policies include:
 - **CachingOptimized** (most common)
 - **CachingDisabled**
 - **CachingOptimizedForUncompressedObjects**
- Cache policy controls: Headers, Cookies, Query strings, Compression support, TTL values (min, default, max)

Exam Tip: Cache Policy controls **what is cached**. Origin Request Policy controls **what is sent to the origin**



CloudFront CheatSheet

Origin Request Policies

- Used when there is a **cache miss**
- Determine **what additional data CloudFront forwards to the origin**
- By default, CloudFront sends:
 - URL path
 - Host header
 - User-Agent
 - Request body (if present)
- Does NOT forward by default:
 - Query strings
 - Cookies
 - Most headers
- Managed policies commonly tested:
 - **AllViewer**
 - **CORS-S3Origin**
 - **AllViewerAndCloudFrontHeaders-2022-06**



CloudFront CheatSheet

Response Header Policies

- Used to **modify headers returned to viewers**
- Common use cases:
 - Security headers (CSP, HSTS, X-Frame-Options)
 - CORS configuration
- AWS-managed policies:
 - **SecurityHeadersPolicy**
 - **CORS-with-preflight**
- Certain headers **cannot be removed**, including:
 - X-Amz-Cf-*
 - X-Cache
 - Transfer-Encoding

Exam Tip: Using response headers for **SPA security hardening**



CloudFront CheatSheet

Securing S3 Origins (OAI vs OAC)

Origin Access Identity (OAI) — Legacy

- Older method to restrict S3 access to CloudFront
- Limitations:
 - No SSE-KMS support
 - Limited HTTP method support
 - Not recommended for new designs
- **Still appears in exam questions**, but usually as the *wrong answer*

Origin Access Control (OAC) — Current Best Practice

- Modern replacement for OAI
- Uses IAM service principals and short-term credentials
- Supports:
 - All HTTP methods
 - SSE-KMS encryption
 - All AWS regions

Exam Tip: **New designs** → **OAC**. **Old architecture** → **possibly OAI**



CloudFront CheatSheet

Origin Shield

- Additional centralized caching layer
- Improves: Cache hit ratio, Origin protection, and Network efficiency
- Best for: High-traffic distributions and Expensive or sensitive origins

Geographic Restrictions

- Restrict content by **country**
- Applies to the **entire distribution**
- Returns **403 Forbidden** for blocked locations

Exam Tip: Geo restriction $\neq$ WAF (WAF works at rule level, Geo restriction is coarse-grained)

Query Strings & Cookies in Caching

- By default, CloudFront:
 - Does NOT cache based on query strings
 - Does NOT forward cookies
- Use:
 - **Cache Policy** $\rightarrow$ include in cache key
 - **Origin Request Policy** $\rightarrow$ forward to origin
 - Common scenario: Language-based content (`?lang=en``, `?lang=fr``)



CloudFront CheatSheet

MIME Types

- CloudFront **does not infer MIME types**
- Browsers rely on `Content-Type` header, not file extension
- Best practice: Set correct Content-Type at upload time (especially for S3)
- Incorrect MIME types can break: JavaScript, CSS, SVG

Cache Invalidation

- Used to expire objects **before TTL**
- Can invalidate: Individual files, Path patterns, Entire cache (expensive)
- Best practice: Invalidate only what changed
- Alternative strategy: **Cache-busting with file hashes** (preferred for static sites)

Default Root Object

- Default is **index.html**
- Controls what is returned for /
- Does NOT apply to subdirectories automatically
- Often tested with: Static website hosting and SPA configurations



CloudFront CheatSheet

HTTPS Enforcement

- CloudFront supports enforcing HTTPS: Viewer Protocol Policy → Redirect HTTP to HTTPS
- Custom domains require **ACM certificate in us-east-1**
- S3 website endpoints **do not support HTTPS**. Use S3 REST endpoint instead

Signed URLs vs Signed Cookies

Signed URLs

- Grant access to **single objects**
- Use cases: Temporary downloads
- Policies: Canned (simple expiry) and Custom (IP, date, restrictions)

Signed Cookies

- Grant access to **multiple objects**
- Ideal for: Paid video libraries and Authenticated user portals
- Requires:
 - CloudFront-Policy
 - CloudFront-Signature
 - CloudFront-Key-Pair-Id



CloudFront CheatSheet

Custom Error Pages

- CloudFront can: Serve custom error pages and Change response codes
- Common SPA pattern: Route all 404s → ``index.html``

Compression

- CloudFront supports automatic compression: **Gzip** and **Brotli**
- Controlled via **Cache Policy**
- Improves: Performance and Bandwidth usage



Simple Queue Service (SQS) CheatSheet

- **Amazon SQS** is a **fully managed message queuing service** that enables **asynchronous communication** between distributed application components.
- It is designed to **decouple producers and consumers**, allowing each to scale independently and fail independently.
- SQS uses a **pull-based model**:
 - Consumers **poll** the queue for messages.
 - It is **not event-driven** or reactive by itself (unlike SNS or EventBridge).
- Messages are **deleted only after successful processing**, which is fundamental to its reliability model.

Common use cases

- Background job processing (image resizing, data transformation)
- Order and workflow processing
- Transactional notifications (email, SMS via downstream services)
- Buffering traffic spikes to protect backend systems



Simple Queue Service (SQS) CheatSheet

Core Message Properties

- **Message Size:** 1 byte - 256 KB
- For messages **larger than 256 KB**, you must use the **Amazon SQS Extended Client Library**, which:
 - Stores the payload in **Amazon S3**
 - Sends a **pointer reference** in the SQS message
 - Supports payloads up to **2 GB**
- This pattern is frequently tested as “**SQS + S3 for large payloads**”

Message Retention Period

- Defines how long SQS stores a message **if it is not deleted**
- Default: **4 days**
- Minimum: **60 seconds**
- Maximum: **14 days**
- If the retention period expires, the message is **permanently deleted**, even if never consumed



Simple Queue Service (SQS) CheatSheet

SQS Queue Types (Critical Exam Topic)

Standard Queues

- **Delivery guarantee: At-least-once delivery**
- Messages **may be delivered more than once**
- Messages **may arrive out of order**
- Provides **best-effort ordering**, not guaranteed ordering
- Designed for **maximum throughput and scalability**
- Supports **nearly unlimited transactions per second**

Exam Tips:

- Consumers **must be idempotent** (safe to process duplicates)
- Use Standard queues when:
 - Ordering is not critical
 - Throughput and scale are more important than precision



Simple Queue Service (SQS) CheatSheet

FIFO Queues

- **Delivery guarantee: Exactly-once processing** (no duplicates)
- **Strict message ordering** is preserved
- Ordering is enforced **within a Message Group**
- **Messages require** `MessageGroupId` and `MessageDeduplicationId` (or content-based deduplication)

Throughput Limits

Without high throughput: **300 messages per second** for send/receive/delete

With **High Throughput FIFO enabled**: Up to **3,000 messages per second per Message Group**

Requires setting: `FifoThroughputLimit = perMessageGroupId`

Important constraints

- You **cannot convert** a Standard queue into a FIFO queue
- FIFO queue names **must end with** `.fifo`
- FIFO queues **do not support per-message timers** (delay applies at queue level only)

Message Group ID (FIFO-specific)

- Messages with the **same MessageGroupId** are processed **in strict order**
- Different Message Group IDs can be processed in **parallel**
- Best practice: Assign a unique Message Group ID per logical workflow or entity (e.g., order ID)



Simple Queue Service (SQS) CheatSheet

Visibility Timeout (Very High Exam Frequency)

- The **visibility timeout** defines how long a message is hidden **after being received**
- Default: **30 seconds**
- Minimum: **0 seconds**
- Maximum: **12 hours (43,200 seconds)**

Why it matters

- Prevents multiple consumers from processing the same message simultaneously
- If the consumer:
 - **Deletes the message** → processing is complete
 - **Fails or times out** → message becomes visible again and can be retried

Classic exam scenario

A long-running job causes messages to be processed twice → **increase visibility timeout**



Simple Queue Service (SQS) CheatSheet

Delay Queues vs Message Timers

Delay Queues

- Delay the delivery of **all new messages** added to the queue
- Default delay: **0 seconds**
- Maximum delay: **15 minutes (900 seconds)**
- Behaviour differs by queue type:
 - **Standard queue**: delay applies only to new messages
 - **FIFO queue**: delay applies to **all messages**

Message Timers

- Delay the delivery of **individual messages**
- Configured using the same `DelaySeconds` attribute
- **Not supported for FIFO queues**
- Useful when each message needs a different delay duration



Simple Queue Service (SQS) CheatSheet

Polling Behavior (Short vs Long Polling)

Short Polling (Default)

- `WaitTimeSeconds = 0``
- Returns immediately, even if the queue is empty
- Can result in: Higher API costs and Empty responses

Long Polling (Best Practice)

- `WaitTimeSeconds > 0`` (up to 20 seconds)
- Waits until: A message arrives, or The timeout expires
- Reduces: Number of API calls, Cost, Empty responses

Exam Tip: If the question mentions **cost optimization** or **efficient polling**, the answer is **Long Polling**

Message Attributes (Metadata)

- Allow you to attach structured metadata to messages
- Supported data types: `String``, `Number``, `Binary``, Custom typed variants (e.g., `Number.short``, `Binary.png``)
- Metadata **does not count as message body**
- Often used for: Routing logic and Filtering (especially with SNS → SQS fanout)



Simple Queue Service (SQS) CheatSheet

Security & Access Control

Queue Encryption

- SQS supports **Server-Side Encryption (SSE)** using:
 - AWS-managed keys, or **AWS KMS customer-managed keys**
 - Encryption applies to Message body and Message attributes

Access Policies

- SQS queues use **resource-based policies**
- Common use case: Allowing an **SNS topic** to send messages to a queue
- Common actions: ``sqs:SendMessage``, ``sqs:ReceiveMessage``, ``sqs:DeleteMessage``

ABAC (Attribute-Based Access Control)

- SQS supports **tag-based access control**
- IAM policies can reference: ``aws:ResourceTag``, ``aws:RequestTag``, ``aws:TagKeys``
- Useful for Environment isolation (prod vs dev) and Least-privilege enforcement



SNS CheatSheet

Pub/Sub Fundamentals (Exam Core Concept)

Publish–Subscribe (Pub/Sub) is a messaging pattern where:

- **Publishers** send messages to a logical channel (a **topic**) instead of directly to consumers.
- **Subscribers** receive messages automatically when something is published to the topic.
- Publishers **do not know** who the subscribers are.
- Subscribers **do not poll**; messages are **pushed** to them.
- SNS uses the terms **messages** and **events** interchangeably.

Why this matters for the exam:

- Pub/Sub is about **decoupling** services.
- SNS is used when **fan-out, event notifications**, or **real-time alerts** are required.
- If ordering, replay, or long-term storage is needed → SNS alone is usually **not enough**.

Amazon SNS is a fully managed, highly available, durable, secure, and **push-based** messaging service

- It is designed to: decouple microservices, broadcast events to multiple subscribers, and deliver notifications quickly and reliably

SNS is NOT a message queue replacement (that's SQS), a streaming platform (that's Kinesis / MSK) or a pull-based service



SNS CheatSheet

SNS Core Components

SNS Topic

- A **logical access point** for communication.
- Publishers send messages **to the topic**.
- Subscribers subscribe **to the topic**, not directly to publishers.
- One topic can deliver messages to **many subscribers across different protocols** simultaneously.
- Topics can be **encrypted using AWS KMS**.

Common publishers include:

- AWS services (S3, CloudWatch, EventBridge, Lambda, etc.)
- Applications using AWS SDK or CLI
- Infrastructure automation tools (CloudFormation, CodePipeline)

Publishers don't care how messages are delivered and don't know who receives the messages

Subscribers define **where** the message goes and **how** it is delivered (protocol)

Each subscription uses **one protocol** and subscribes to **one topic**



SNS CheatSheet

SNS Topic Types (VERY IMPORTANT)

SNS supports **two topic types**, and exam questions often test the differences.

Standard Topics

- **High throughput**
- **At-least-once delivery**
- **Possible duplicate messages**
- **No ordering guarantees**

Typical use cases:

- Alerts
- Monitoring notifications
- Fan-out to multiple systems where strict order is not required



SNS CheatSheet

FIFO Topics

- Topic name **must end with** `.fifo`
- **Exactly-once delivery**
- **Strict ordering** within a message group
- Lower throughput than standard topics

Key requirements: Message Group ID (controls ordering) and Deduplication ID (or content-based deduplication)

Typical use cases:

- Financial transactions
- Ordered workflows
- Systems where duplicates or reordering are unacceptable

Exam Tip: Not all AWS services can publish to FIFO SNS topics.



SNS CheatSheet

SNS Destinations (Subscribers)

Application-to-Application (A2A)	Application-to-Person (A2P)
Used for system integration: • AWS Lambda • Amazon SQS • HTTP / HTTPS endpoints (webhooks) • Amazon Data Firehose	Used for human notifications: • Email (plain text or JSON) • SMS (text messages) • Mobile push notifications • AWS Chatbot (Slack / Amazon Chime) • Third-party integrations (e.g., PagerDuty)

Exam Tip: If the question mentions fan-out to multiple systems, SNS is almost always involved.

Publishing Messages

Standard Publishing

- Applications publish messages programmatically using:
 - AWS SDKs
 - AWS CLI
 - AWS services (via native integration)

Messages are published to a **topic ARN**.



SNS CheatSheet

Large Message Support (Frequently Tested Detail)

SNS has a **256 KB message size limit**.

To publish larger messages:

- Use the **Amazon SNS Extended Client Library** (Java or Python)
- Payload is stored in **Amazon S3**
- SNS message contains a **reference** to the S3 object
- Maximum supported payload size: **up to 2 GB**

Exam Tip: SNS itself does **not** store large payloads—S3 is used behind the scenes.

Message Attributes

- SNS supports **message attributes** for metadata: ``String``, ``String.Array``, ``Number``, ``Binary``
- Attributes are:
 - Used for **filtering**
 - Not part of the message body
 - Evaluated **before delivery**

Common exam scenario: Use message attributes to route messages **without duplicating topics**.



SNS CheatSheet

Batch Publishing

- ``PublishBatch`` API allows sending **up to 10 messages per request**.
- Reduces API calls
- Can significantly **reduce SNS costs**

Message Structure (Protocol-Specific Messages)

SNS can deliver **different messages to different protocols**.

How it works:

- Set ``MessageStructure`` to ``json``
- Provide protocol-specific payloads

Example protocols: ``default``, ``email``, ``sms``, ``sqs``, ``http``, ``https``

Exam Tip: SNS can format messages **automatically per subscriber protocol**.

Subscriptions

To receive messages, a **subscription is required**.

Supported protocols include: HTTP / HTTPS (webhooks), Email (plain text), Email-JSON, Amazon SQS, AWS Lambda, SMS, Mobile push (platform application endpoints)

Important constraint: A subscription can only use **one protocol** and **one topic**.



SNS CheatSheet

Filter Policies (Highly Exam-Relevant)

SNS Filter Policies allow:

- Selective message delivery
- Reduced downstream processing
- Smarter fan-out

Filter Policy Scope

- **MessageAttributes** (most common)
- **MessageBody** (must explicitly set `FilterPolicyScope=MessageBody``)

Supported Filtering Logic

- AND / OR logic
- Numeric comparisons (exact, range, anything-but)
- String comparisons (exact, prefix, suffix, ignore-case)
- IP address matching

Exam Tip: If you see “only some subscribers should receive the message” → Filter Policy.



SNS CheatSheet

Message Data Protection (Newer Feature – Know the Basics)

SNS supports **Message Data Protection** for **standard topics only**.

Capabilities:

- Detect sensitive data such as PII and PHI
- Uses predefined or custom **data identifiers**
- Actions: Audit (log findings), De-identify (mask or redact), Deny (block message)
- Findings can be sent to: CloudWatch Amazon S3 Amazon Data Firehose

Raw Message Delivery

- Prevents SNS from wrapping messages in JSON metadata and sends the message **as-is**
- Applies to Amazon SQS, Amazon Data Firehose, HTTP / HTTPS endpoints
- **Why it matters:** Simplifies downstream processing and avoids unnecessary parsing

Delivery Retry Policy

SNS automatically retries failed deliveries. Key points:

- Each protocol has a **default retry behavior**
- Retries happen in phases: Immediate, Pre-backoff, Backoff (exponential), Post-backoff
- When retries are exhausted message is **discarded**, unless a **Dead-Letter Queue (DLQ)** is configured

Only **HTTP/HTTPS subscriptions** allow **custom delivery policies**.



SNS CheatSheet

Dead-Letter Queues (DLQ)

SNS supports DLQs using **Amazon SQS**.

Rules:

- Standard SNS topic → Standard SQS queue
- FIFO SNS topic → FIFO SQS queue
- Topic and queue types **must match**

DLQs:

- Capture messages that fail delivery
- Improve reliability and troubleshooting
- Are attached at the **subscription level**

Exam Tip: SNS + SQS DLQ is a very common resilience pattern.

Mobile Push Notifications

- SNS can send push notifications directly to mobile apps via: Apple Push Notification Service (APNs), Firebase Cloud Messaging (FCM), and Other platform notification services
- Messages can appear as: Alerts, Badges, Sounds
- This requires Platform applications or Platform endpoints



RDS CheatSheet

Amazon Relational Database Service (RDS) is a **fully managed relational database service** that handles:

- Operating system patching
- Database engine patching (unless using RDS Custom)
- Backups and restores
- High availability (Multi-AZ)
- Failover detection
- Monitoring and metrics

You still manage:

- Database schema
- Indexes
- Queries
- Users and permissions
- Parameter tuning (via parameter groups)

Exam Tip: If the question says “***you don’t want to manage OS patching or backups***”, the answer is **RDS** (or **Aurora**, which is still RDS).



RDS CheatSheet

Supported RDS Engines (Know the Differences)

Open-Source Engines	Commercial / Enterprise Engines
• MySQL • Widely used, simple architecture • Logical replication for read replicas • Common for web workloads • MariaDB • Community-driven fork of MySQL • Drop-in compatible in most cases • Similar replication and performance characteristics • PostgreSQL • Feature-rich (JSONB, extensions, advanced indexing) • More complex but more powerful • Strong consistency and query planner	• Microsoft SQL Server • Tight integration with Active Directory and Kerberos • Licensing required (license-included or BYOL) • Supports IAM authentication and Kerberos • Oracle • Enterprise-grade, complex licensing • Supports Data Guard, Kerberos, and advanced features • Used heavily in legacy enterprise systems • IBM Db2 • Enterprise workloads • BYOL model • Less common on the exam but still testable



RDS CheatSheet

Aurora (Still RDS)

- MySQL-compatible and PostgreSQL-compatible
- Automatically scales storage
- Up to **15 read replicas**
- Faster failover than standard RDS
- Separate compute and storage layers

Exam Tip: Aurora ≠ “another service” — it is RDS, just architected differently.

Encryption at Rest

- Uses **AWS Key Management Service (KMS)**
- Encrypts: Database storage, Automated backups, Manual snapshots, Read replicas
- **Must be enabled at DB creation time**
- Cannot encrypt an existing unencrypted DB directly. You must snapshot → restore with encryption

Encryption in Transit

- Enabled by default using **TLS**
- Happens automatically via the RDS DNS endpoint

Exam Tip: If asked “can I enable encryption later?” → No, only via restore.



RDS CheatSheet

Backups (Highly Tested Topic)

Automated Backups	Manual Snapshots
- Retention: 0–35 days - Enables Point-in-Time Recovery (PITR) with 1-second precision - AWS: - Takes a daily snapshot - Continuously captures transaction logs - Stored in Amazon S3 - Taken during a configurable backup window - Free (included in pricing) - Setting retention to 0 disables automated backups	- User-initiated - Persist even after DB deletion - Can be: - Copied cross-region - Shared cross-account - Exported to S3 Storage is billed separately

Exam Tip:

- Automated backups = short-term recovery
- Manual snapshots = long-term retention & migrations



RDS CheatSheet

Restore Behavior (Critical Exam Detail)

- Restoring **always creates a new DB instance**
- Original DB is untouched
- Applies to: Automated backups, Manual snapshots, and PITR restores

Exam Tip: Restore time impacts **RTO** — RDS restore is not instant.

DB Subnet Groups

- A **DB subnet group** is a set of subnets within a VPC
- Must include **at least two Availability Zones**
- Determines **where RDS can be placed**
- For public access, **all subnets must be public**

Exam Tip: Subnet groups $\neq$ security groups. Subnet groups decide **placement**, not access.



RDS CheatSheet

Multi-AZ (High Availability, Not Scaling). What Multi-AZ Does:

- Creates a **synchronous standby** in another AZ
- AWS automatically detects failure, promotes standby to primary, and updates DNS endpoint

Key Characteristics

- Synchronous replication
- Only primary is writable
- Standby is not accessible
- Automated backups taken from standby

Exam Tip: Multi-AZ improves **availability, not read performance.**

Read Replicas (Scaling, Not HA). Core Concepts:

- Asynchronous replication
- Used to reduce read contention
- Each replica has its own DNS endpoint
- Can be: Same AZ, Cross-AZ, or Cross-region

Limits: MySQL / MariaDB / PostgreSQL = **up to 5**; Aurora = **up to 15**

Important Limitations: No automatic failover, Promotion breaks replication, Requires automated backups enabled

Exam Tip: Multi-AZ = failover, Read replicas = read scaling



RDS CheatSheet

Multi-AZ vs Read Replicas (Compare Table)

Feature	Multi-AZ	Read Replica
Replication	Synchronous	Asynchronous
Purpose	High availability	Read scaling
Failover	Automatic	Manual
Read traffic	No	Yes
Writable	Primary only	No (unless promoted)

DB Instance Classes

- Always start with ``db.``
- Categories:
 - **General purpose**: db.m6i, db.m7g
 - **Memory optimized**: db.r6g, db.r7g
 - **Burstable**: db.t3, db.t4g
- Architecture: **Graviton (g)** = ARM-based, cheaper, more efficient and **Intel (i)** = x86 compatibility

Exam Tip: Cost optimization questions often favour **Graviton**.



RDS CheatSheet

Storage Types

Supported

- **gp3 / gp2** (General Purpose SSD)
- **io1 / io2 Block Express** (Provisioned IOPS)

Deprecated / Not Recommended

- Magnetic storage (legacy)

Key Rules

- Storage can **only be increased**
- Cannot be reduced
- To shrink storage → restore to new DB

Performance Insights

- Enabled by default
- Retains **7 days** free
- Up to **2 years** with additional cost
- Helps identify: SQL bottlenecks, Wait events, CPU and I/O pressure

Exam Tip: Performance Insights ≠ CloudWatch metrics (it's deeper).



RDS CheatSheet

RDS Proxy (Very Exam-Relevant)

- Manages database connections
- Prevents connection exhaustion
- Ideal for **AWS Lambda** and short-lived connections
- Similar to PgBouncer (Postgres)

Exam Tip: If Lambda + RDS + connection issues → **RDS Proxy**

Authentication Options

IAM Database Authentication

- Supported by: MySQL, MariaDB, PostgreSQL
- Uses temporary tokens (15 min)
- Eliminates stored DB passwords

Kerberos / Active Directory

- Integrates with: AWS Managed Microsoft AD and On-prem AD
- Supported by: SQL Server, PostgreSQL, MySQL, and Oracle

Exam Tip: Enterprise SSO → Kerberos
Cloud-native auth → IAM auth



RDS CheatSheet

Secrets Manager Integration

- Automatically rotates master password
- Default rotation: **7 days**
- Does **not work with**: SQL Server, RDS Custom, or Blue/Green deployments

RDS Blue/Green Deployments

- Creates a synchronized staging environment
- Used for: Schema changes, Engine upgrades, and Feature testing
- Enables near-zero downtime cutover

Exam Tip: Blue/Green = **safe production changes**, not HA.

RDS Extended Support

- Lets you run **unsupported major versions**
- Additional cost
- Provides **critical & high CVE patches**
- Available up to **3 years**
- After that → forced upgrade

Exam Tip: Extended Support ≠ permanent solution — it's a grace period.



Amazon Aurora CheatSheet

- **Amazon Aurora** is a **fully managed relational database service** designed for high performance and availability, while remaining compatible with open-source engines.
- It supports **MySQL-compatible** and **PostgreSQL-compatible** engines, allowing existing applications to migrate with minimal changes.
- Aurora is architected differently from standard RDS engines:
 - Compute (DB instances) and storage are **decoupled**
 - Storage is **distributed, replicated, and self-healing**
- AWS advertises:
 - **Up to 5× the performance of standard MySQL**
 - **Up to 3× the performance of standard PostgreSQL**
- Designed to deliver **enterprise-grade performance and availability** at a cost closer to open-source databases.

Exam Tip: If a question mentions **high performance + MySQL/Postgres compatibility + automatic scaling storage**, Aurora is almost always the correct answer.



Amazon Aurora CheatSheet

Aurora Storage Architecture

- An Aurora cluster:
 - Starts at **10 GB** of storage
 - **Automatically scales** in **10 GB increments**
 - Grows up to **128 TB** (engine/version dependent)
- Storage is **shared across all instances in the cluster**
- No need to provision IOPS or storage size in advance

Multi-AZ Durability

- Data is replicated **6 times across 3 Availability Zones. 2 copies per AZ**
- Fault tolerance:
 - Can lose **up to 2 copies** without impacting **writes**
 - Can lose **up to 3 copies** without impacting **reads**

Self-Healing Storage

- Aurora continuously:
 - Scans data blocks for errors
 - Automatically repairs corrupted blocks
- No user intervention required

Exam Tip: Aurora achieves high availability primarily through **storage replication**, not just read replicas.



Amazon Aurora CheatSheet

Aurora Security (Frequently Tested)

- **Encryption at rest**
 - **Always enabled**
 - Cannot be turned off
 - Uses **AWS KMS** (AWS-managed or customer-managed keys)
- **Encryption in transit**
 - Uses **TLS/SSL**
 - Encryption terminates at the database engine
- Supports:
 - IAM authentication (engine-dependent)
 - VPC security groups
 - Private subnets



Amazon Aurora CheatSheet

Aurora Cluster Architecture: Writer vs Readers

Writer Instance

- Exactly **one writer per Aurora cluster**
- Handles **all write operations** and can also serve **read traffic**
- If the writer fails, Aurora automatically **promotes a reader to writer**
- **Scales vertically only** (change instance size)

Reader Instances (Aurora Replicas)

- Up to **15 readers per cluster**
- Used for:
 - Read-heavy workloads
 - Analytics and reporting
 - Horizontal read scaling
- Can be:
 - Load-balanced using the **reader endpoint**
 - Promoted automatically during failover

Exam Tip: Aurora readers are **failover targets**, unlike standard RDS read replicas which require manual promotion.



Amazon Aurora CheatSheet

Aurora Provisioned (Default Deployment Model)

- **Aurora Provisioned** is the standard, always-on deployment model.
- You explicitly choose: Instance class (e.g., db.r6g.large) and Number of reader instances
- Billed for **Instance hours** and **Consumed storage**
- Best for Predictable workloads, Steady performance requirements, and Long-running production databases

Aurora Serverless v2 (Exam-Relevant and Current)

Important Deprecation Notice

- **Aurora Serverless v1 is deprecated and no longer supported after December 31, 2024**
- **Aurora Serverless v2** is the only serverless Aurora option you should reference on the exam

How Serverless v2 Works

- Automatically scales compute capacity, not storage
- Scaling is **Fine-grained** and Near-instant (seconds)
- Uses **Aurora Capacity Units (ACUs)**:
 - **1 ACU $\approx$ 2 GiB memory + CPU + networking**
 - Range: **0.5 to 128 ACUs**
- Does **not scale to zero**. Minimum capacity is **0.5 ACUs**



Amazon Aurora CheatSheet

Configuration Details

- You must define **Minimum ACU** and **Maximum ACU**
- Writer and readers use the instance class: ``db.serverless``

Billing

- Charged per **ACU-second**
- More granular and cost-efficient for spiky workloads

Best Use Cases

- Highly variable traffic
- Unpredictable workloads
- Applications that need rapid scaling without downtime



Amazon Aurora CheatSheet

Aurora Serverless v2 vs Aurora Provisioned (Key Exam Comparison)

Feature	Aurora Serverless v2	Aurora Provisioned
Scaling	Automatic, fine-grained	Manual
Scaling Speed	Seconds	Requires maintenance
Capacity Model	0.5-128 ACUs	Fixed instance sizes
Billing	ACUs per second	Instance hours
Start/Stop	Responsive, automatic	Manual
Best For	Spiky or unpredictable loads	Steady workloads

Exam Tip: If the question emphasizes **cost efficiency for unpredictable traffic**, choose **Aurora Serverless v2**.



Amazon Aurora CheatSheet

Aurora Global Database

- A **multi-region Aurora deployment** designed for Global low-latency reads and Disaster recovery
- Architecture:
 - **1 primary region** (writes)
 - Up to **5 secondary regions** (read-only)

Replication

- Storage-level replication
- Typical replication latency: **< 1 second**

Failover

- Manual or planned promotion of a secondary region
- Used for Global applications and Regional disaster recovery strategies

Exam Tip: Aurora Global Database is about **cross-region availability**, not multi-AZ (that's already built-in).



Amazon Aurora CheatSheet

Aurora and the RDS Data API

- Allows you to run SQL queries over **HTTPS**
- No persistent database connections required
- Commonly used with **AWS Lambda** and Event-driven architectures

Key Limitations

- Must be **explicitly enabled** on the cluster
- **Multi-statement SQL is not supported**
- Limited data-type support
- CloudTrail: Data API calls are **data events** and **not logged by default**

Query Editor

- The Aurora Query Editor in the AWS Console is simply a UI wrapper around the **RDS Data API**



Amazon Aurora CheatSheet

Babelfish for Aurora PostgreSQL

- Enables **SQL Server-compatible applications** to run on **Aurora PostgreSQL**
- Supports: **T-SQL** over port **1433** and **PostgreSQL protocol** over port **5432**

Migration Benefits

- Minimal application code changes
- No need to rewrite database drivers
- Useful for SQL Server modernization projects

Current Limitations (Exam-Relevant). Babelfish does **not** support:

- RDS Blue/Green Deployments
- AWS IAM database authentication
- Database Activity Streams
- PostgreSQL logical replication
- RDS Data API
- RDS Proxy
- Kerberos authentication
- Query Editor

Exam Tip: Babelfish is for **compatibility**, not feature parity with SQL Server.



Amazon Aurora CheatSheet

High-Yield Aurora Exam Scenarios

- **Choose Aurora when you need:**
 - MySQL/Postgres compatibility
 - Automatic storage scaling
 - High availability without manual replication
- **Choose Serverless v2 when:**
 - Traffic is unpredictable
 - You want near-instant scaling
- **Choose Global Database when:**
 - You need multi-region reads or disaster recovery
- **Remember:**
 - Aurora $\neq$ standard RDS architecture
 - Aurora replicas can be **failover targets**
 - Serverless v1 is **deprecated**



Elastic Load Balancing (ELB) CheatSheet

Elastic Load Balancing (ELB) is a managed service in AWS that automatically distributes incoming traffic across multiple targets (EC2, ECS, EKS, Lambda, IPs) to improve: **High availability, Fault tolerance, Scalability, Security**

On the exam, ELB questions usually test which load balancer to choose, how traffic is routed, and how ELB integrates with other AWS services.

Types of Load Balancers (You Must Know All 4)

1. Application Load Balancer (ALB) — Layer 7

- **Primary use case: HTTP/HTTPS applications**
- Operates at **OSI Layer 7 (Application layer)**.
- Supports **advanced request routing** based on: Host headers, Path patterns, HTTP headers, HTTP methods, Query strings, Source IP
- Uses **listeners + rules + target groups**.
- Fully integrates with: **AWS WAF** (web application firewall), **AWS Certificate Manager (ACM)** for HTTPS, **Amazon Cognito** or OIDC providers for authentication
- Supports **HTTP/2** and **WebSockets**.
- **No static IPs** (accessed via DNS name only). If a static IP is required → put **NLB in front of ALB**.

Use Cases: Microservices, Containerized workloads (ECS, EKS, Fargate), Web apps with multiple routes or domains



Elastic Load Balancing (ELB) CheatSheet

2. Network Load Balancer (NLB) — Layer 4

- **Primary use case: extreme performance and static IPs**
- Operates at **OSI Layer 4 (TCP/UDP)**
- Handles **millions of requests per second with ultra-low latency**.
- **Preserves the client source IP** by default.
- Provides **static IP addresses** (per AZ).
- Supports TCP, UDP, and TLS listeners.
- Can forward traffic to: EC2, IP addresses, ALBs (common pattern)

Use Cases: Real-time systems (gaming, financial trading), IoT backends, Legacy TCP/UDP applications

3. Gateway Load Balancer (GWLb) — Traffic Inspection

- **Primary use case: security and network appliances**
- Routes traffic to **virtual appliances** (firewalls, IDS/IPS).
- Uses **GENEVE protocol**.
- Operates transparently before traffic reaches its destination.
- Frequently used in **security architectures**.

Exam Tip: If the question mentions **firewalls, packet inspection**, or **inline security appliances**, the answer is almost always **GWLb**.



Elastic Load Balancing (ELB) CheatSheet

4. Classic Load Balancer (CLB) — Legacy

- **Not recommended for new architectures**
- Legacy service (pre-ALB/NLB).
- Can operate at **Layer 4 or Layer 7**, but **not both at once**.
- **No target groups** (instances are registered directly).
- Limited routing capabilities.
- Still appears on the exam **only for legacy scenarios**.

Exam Tip: If the question says “existing legacy application” → CLB. Otherwise → **ALB or NLB**

Core ELB Architecture (Very Testable)

- **Listeners** define protocol + port (e.g., HTTP:80, HTTPS:443) and Incoming traffic is evaluated against listeners first.
- **Rules (ALB only)**
 - Determine **how traffic is routed**.
 - Evaluated in **priority order (lowest number first)**.
 - Consist of: Conditions (match logic), Actions (what to do), Priority
- **Target Groups**
 - Logical grouping of targets: EC2 instances, IP addresses, Lambda functions (ALB only)
 - Health checks are configured **at the target group level**.
- CLB exception: CLB **does not use target groups**



Elastic Load Balancing (ELB) CheatSheet

Health Checks (High-Yield Exam Topic)

- ELB continuously monitors target health.
- Targets are marked **InService** or **OutOfService**.
- **ELB never terminates instances** — it only **stops routing traffic**.
- For **ALB and NLB**, health checks live on the **target group**, not the load balancer.
- Typical health check parameters: Protocol (HTTP, HTTPS, TCP), Path (ALB only), Interval and timeout, Healthy/unhealthy thresholds, Expected success codes (e.g., HTTP 200)

Sticky Sessions (Session Affinity)

Sticky sessions ensure that a client continues to be routed to the **same target**.

- Supported by **ALB and CLB**.
- Configured **on the target group**, not per instance.
- Implemented using **cookies**.
- Useful when session state is stored **locally on the instance**.
- **WebSockets are inherently sticky**.
- **Weighted target groups do NOT guarantee stickiness**.
- Application-based cookies **do not work with weighted target groups**.
- ALB uses the **Expires** attribute (not Max-Age).
- Cookie names ``AWSALB``, ``AWSALBAPP``, ``AWSALBTG`` are **reserved**.



Elastic Load Balancing (ELB) CheatSheet

Cross-Zone Load Balancing (Frequently Tested)

Evenly distributes traffic across **all registered targets in all enabled AZs**, not just the AZ that received the request.

Defaults:

- **ALB** → Enabled by default (**cannot be disabled**)
- **NLB** → Enabled by default (**can be disabled**)
- **CLB** → Disabled by default
- Why it matters: Improves fault tolerance, Prevents uneven load distribution, Can impact inter-AZ data transfer costs

X-Forwarded-For (XFF) Header

When using ALB:

- The **client's original IP address** is stored in the **X-Forwarded-For** HTTP header.
- Critical for:
 - Logging
 - Security rules
 - Application-level IP filtering

Exam Tip: If the app "can't see the client IP" behind ALB → use **X-Forwarded-For**



Elastic Load Balancing (ELB) CheatSheet

ALB Rules & Request Routing (Big Exam Area)

Rule Limits (Know These)

- 100 rules per ALB
- 5 condition values per rule
- 5 wildcards per rule
- 5 weighted target groups per rule

Rule Action Types

- **forward** → Send traffic to one or more target groups
- **redirect** → HTTP 301 (permanent) or 302 (temporary)
- **fixed-response** → Return static HTTP responses
- **authenticate-cognito** → Authenticate using Cognito
- **authenticate-oidc** → **Authenticate using an OIDC provider**

Protocol limitation:

- **For HTTP/2 or gRPC, only forward actions are allowed.**



Elastic Load Balancing (ELB) CheatSheet

ALB Condition Types (Layer 7 Power)

You can route traffic based on:

- **host-header** (e.g., api.example.com)
- **path-pattern** (e.g., /images/*)
- **http-header** (e.g., User-Agent)
- **http-request-method** (GET, POST, etc.)
- **query-string** (?version=v1)
- **source-ip** (CIDR-based filtering)

Exam Tip: If routing logic depends on HTTP details, the answer is ALB

Target Registration Options

Supported targets:

- **EC2 instances** (instance ID)
- **IP addresses** (IPv4 or IPv6)
 - Required for **Fargate**
- **Lambda functions** (ALB only)
- **Another ALB** (via NLB or ALB chaining)

Exam Tip: Lambda targets **do not need to be in a VPC**. Only **ALB target groups** can target Lambda.



Elastic Load Balancing (ELB) CheatSheet

Deregistration Delay (Connection Draining)

Allows **in-flight requests to complete** before removing a target.

- Key values:
 - Minimum: 1 second
 - Maximum: 3600 seconds (1 hour)
 - Default: 300 seconds (5 minutes)
- Differences:
- **ALB & CLB** → Configurable
- **NLB** → Not configurable

Trust Stores & mTLS (Advanced but Testable)

- **ALB Trust Stores** support **mutual TLS (mTLS)** authentication.
- Uses **X.509 certificates**.
- Common in high-security enterprise environments.



IAM CheatSheet

IAM Overview

- Create and manage AWS users and groups
- Use permissions to allow or deny access to AWS resources
- **IAM Users:** End users logging into the console or interacting with AWS resources
- **IAM Groups:** Group users to share permission levels (e.g., Administrators, Developers, Auditors)
- **IAM Roles:** Grant AWS resources permissions to specific AWS API actions
 - Associate policies with a role, then assign it to an AWS resource
- **IAM Policies:** JSON documents granting permissions to users, groups, or roles
 - Types: Managed Policies, Customer Managed Policies, Inline Policies

Policy Elements

- **Version:** Policy language version (latest is 2012-10-17)
- **Statement:** Container for policy elements
- **Effect:** Allow or Deny
- **Action:** List of actions the policy allows or denies
- **Principal:** Account, user, role, or federated user
- **Resource:** The resource the action applies to
- **Condition:** Circumstances under which the policy grants permission



IAM CheatSheet

Principle of Least Privilege (PoLP)

- Provide the least amount of permissions needed
- Concepts: Just-In-Time (JIT), Just-Enough-Access (JEA), Risk-based adaptive policies

AWS Account Root User

- Special user with full access; cannot be deleted
- Root user should not be used for daily tasks
- Strongly recommended to enable MFA for the root user

IAM Password Policy

- Set minimum password requirements and rotation policies

IAM Access Keys

- Allow programmatic interaction with AWS services
- Two access keys per user

IAM Multi-Factor Authentication (MFA)

- Additional layer of security
- Can be enforced by administrators



IAM CheatSheet

IAM Temporary Security Credentials

- Temporary credentials for roles and identity federation
- Last from minutes to hours
- Basis for roles and identity federation

IAM Identity Federation

- Types: Enterprise (SAML, Custom Federation broker), Web (Amazon, Facebook, Google, OIDC)
- Allows linking of electronic identities across multiple systems

AWS Security Token Service (STS)

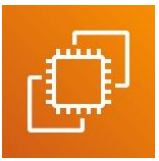
- Request temporary, limited-privilege credentials
- Key API actions: AssumeRole, AssumeRoleWithSAML, AssumeRoleWithWebIdentity, etc.

IAM Cross-Account Roles

- Grant users from different AWS accounts access to your resources
- Role creation involves a policy granting sts:AssumeRole access

AWS Single Sign-On (SSO)

- Create or connect workforce identities in AWS and manage access centrally
- Identity sources: AWS SSO, Active Directory, SAML 2.0 IdP



EC2 CheatSheet

Introduction to EC2:

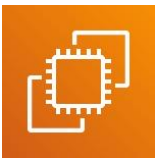
- Highly configurable virtual server
- Resizable compute capacity; launch instances in minutes
- Core component for AWS services

Instance Selection:

- Choose OS via Amazon Machine Image (AMI)
- Instance Types:
 - General Purpose: balance of compute, memory, and networking (e.g., T2, T3, M5)
 - Compute Optimized: high-performance processors (e.g., C5)
 - Memory Optimized: fast performance for large data sets (e.g., R5, X1)
 - Storage Optimized: high read/write access to large datasets (e.g., I3, D2)
 - Accelerated Computing: hardware accelerators (e.g., P3, G4)

Pricing and Configurations:

- Pricing varies by instance type and size (e.g., t2.nano \$0.0065/hour)
- Configure with storage options: EBS, EFS
- Add security: Security Groups, Key Pairs, IAM Roles



EC2 CheatSheet

Cloud-Init and User Data:

- Cloud-init for cross-platform cloud instance initialization
- User Data to run scripts at first boot (e.g., install web servers)
- Meta-data and User-data for instance configuration

Instance Metadata:

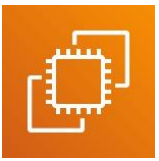
- Access metadata via special endpoint (IPv4: <http://169.254.169.254/latest/meta-data/>)
- Two versions: IMDSv1 (request/response), IMDSv2 (session-oriented)
- Categories include ami-id, block-device-mapping, elastic-gpus, etc.

Instance Lifecycle:

- States: pending, running, stopping, stopped, shutting-down, terminated
- Actions: launch, stop, start, terminate, reboot, retire, recover

Instance Types Naming Convention:

- Instance family, generation, processor family, additional capabilities, size (e.g., c7gn.xlarge)



EC2 CheatSheet

Burstable Instances:

- Handle bursts of higher CPU utilizations
- Modes: Standard (baseline with bursts), Unlimited (sustain high CPU)
- Types: T2, T3, T3a, T4g

Instance Profile:

- Reference to IAM role for instance access without AWS credentials
- Single IAM role per Instance Profile; change requires reboot

Placement Groups:

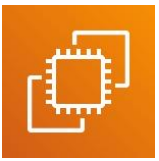
- Types: Partition, Cluster, Spread
- Optimize instance placement for communication, performance, durability

System Logs and Console:

- Logs delivered to CloudWatch or viewed in EC2 Management Console
- Instance Console Screenshot for troubleshooting boot issues

Connectivity Options:

- EC2 Instance Connect, Sessions Manager, SSH Client, Serial Console, Fleet Manager Remote Desktop



EC2 CheatSheet

Hostnames:

- Hostname types: IP Name (IPv4), Resource Name (IPv6)
- Format varies by region and instance type

Default User Names:

- Varies by AMI (e.g., ec2-user for Amazon Linux, ubuntu for Ubuntu)

Amazon Linux:

- AWS managed Linux distribution (based on CentOS and Fedora)
- Versions: AL1 (EOL Dec 2023), AL2, AL2023
- Package management: yum for AL2, dnf for AL2023

Amazon Linux Extras and EPEL:

- AL2: use Amazon Linux Extras for additional packages
- EPEL for broader range of packages

Differences between AL2 and AL2023:

- AL2: Python 2.7, OpenSSL 1.0.2, gp2 volumes, Cronie installed
- AL2023: Python 3, OpenSSL 3, gp3 volumes, SELinux enabled by default, Cronie not installed



AMI CheatSheet

AMI Information:

- Template for root volume (EBS Snapshot or Instance Store template)
- Launch permissions for AWS accounts
- Block device mapping for attached volumes

Use Cases:

- Incremental changes to OS, application code, system packages
- Routine patching with security updates
- Managed revisions with Launch Configurations or Templates

AWS Marketplace:

- Purchase subscriptions to vendor-maintained AMIs
- Popular security-hardened AMIs (e.g., Center of Internet Security (CIS))

Choosing an AMI:

- Consider region, OS, architecture (32/64-bit), launch permissions, root device volume (Instance Store or EBS)

Boot Modes:

- **Legacy BIOS:** Traditional, no secure boot, for legacy OS/software
- **UEFI:** Modern, supports secure boot, faster startups, larger drives, pre-boot environment with UI and network capabilities



AMI CheatSheet

Elastic Network Adapter (ENA):

- Supports up to 100 Gbps network speeds
- Used in Nitro system instances for enhanced networking

Root Device Type:

- EBS-backed: Storage independent, data persists through stops/terminations
- Instance store-backed: Native volumes, data lost on stop/termination

Creating and Copying AMIs:

- Create from running or stopped EC2 instances
- Copy AMIs, including across regions, with optional encryption

Store and Restore AMIs:

- Store in S3 for long-term storage, lifecycle management, disaster recovery, auditing/compliance
- Restore from S3 for cross-AWS partition copying



AMI CheatSheet

Deprecate, Disable, and Deregister:

- **Deregister:** Prevent new instances, does not delete snapshots
- **Deprecate:** Mark a date to stop usage
- **Disable:** Temporarily prevent usage, can be re-enabled

Sharing AMIs:

- **Public:** Any AWS account can launch
- **Explicit:** Specific AWS accounts/organizations can launch
- **Implicit:** Only the owner can launch
- Can be sold in the AWS Marketplace

Virtualization Types:

- **Hardware Virtual Machine (HVM):** Hardware-assisted, supports unmodified OS, higher performance for direct hardware access
- **Paravirtualization (PV):** Software-assisted, requires OS modification, historically better performance but now less common

Auto Scaling Groups (ASG) CheatSheet

Introduction to ASG

- ASG contains EC2 instances managed as a group for automatic scaling.
- Automatic scaling methods: Capacity Settings, Manual Scaling, Health Check Replacements, Scaling Policies.
- ASGs work with ECS and EKS, but not Fargate.

ASG Use Case

1. Traffic hits the domain.
2. Route53 directs to load balancer.
3. Load balancer passes traffic to the target group.
4. Target group associated with ASG sends traffic to instances.
5. ASG Scaling Policy checks instance capacity.
6. Scaling Policy launches a new EC2 instance if needed.

Capacity Settings

- **Min Size:** Minimum running EC2 instances.
- **Max Size:** Maximum allowed EC2 instances.
- **Desired Capacity:** Ideal number of EC2 instances.
- Manual scaling involves changing these values.

Auto Scaling Groups (ASG) CheatSheet

Health Check Replacements

- **ELB Health Check:** ASG uses ELB health checks.
- **EC2 Health Check:** ASG replaces instances failing EC2 status checks.

ELB Integration

- ALB, NLB, GWLB are indirectly associated via Target Groups.
- Classic Load Balancers are directly associated with ASG.

Dynamic Scaling Policy

- **Adjustment Types:** ChangeInCapacity, ExactCapacity, PercentChangeInCapacity.
- **Policies:** Simple Scaling, Step Scaling, Target Tracking.

Simple Scaling Policy

- Adjust capacity when CloudWatch Alarm is triggered.
- Recommended to set a cooldown period.
- Prefer Step or Target Tracking policies.

Step Scaling Policy

- Adjust capacity at different thresholds when CloudWatch Alarm is repeatedly triggered.

Auto Scaling Groups (ASG) CheatSheet

Target Tracking Scaling Policy

- Scales capacity based on a target metric value (e.g., `ASGAverageCPUUtilization`).
- Automatically creates CloudWatch alarms.

Predictive Scaling Policy

- Analyzes historical load data to predict and scale based on traffic patterns.
- Uses last 14 days of data for hourly forecast.

Termination Policies

- **Predefined policies:** Default, AllocationStrategy, OldestLaunchTemplate, etc.
- Custom termination policies via Lambda Function.

Instance Refresh

- Replace instances to apply new configuration changes.
- **Features:** instance warmup, Min/Max healthy percentages, checkpoints, auto rollback.

Instance Scale-In Protection

- Prevents instances from being terminated.
- Can be enabled or disabled at creation or anytime.

Auto Scaling Groups (ASG) CheatSheet

Launch Template

- Includes parameters: AMI, Instance Type, Key Pair, Security Groups, etc.
- Required to make ASG work.
- Use SSM Parameter to avoid hardcoding ImageID.

Lifecycle Hooks

- Perform custom actions when an instance is pending or terminating.
- Notifies SQS or SNS, use `complete_lifecycle_action` to continue lifecycle.

Warm Pools

- Decrease latency for apps with long boot times.
- Instances in warm pool: Stopped, Running, or Hibernated.
- Use Lifecycle hooks for wait state.

Detach / Attach Instances

- Detach makes an instance independent.
- Attach criteria: running state, existing AMI, etc.
- Detach criteria: InService ASG state.

Auto Scaling Groups (ASG) CheatSheet

Suspend / Resume Processes

- Suspend processes for investigation or troubleshooting.
- Supports processes like Launch, Terminate, HealthCheck, etc.

Troubleshooting

- Failures: EC2 Instance Launch Failures, AMI Issues, Load Balancer Issues, Capacity Limits.
- Use AWS CLI to find exact error.



Route 53 CheatSheet

- **Route 53:** A Domain Name Service (DNS) integrated with AWS. Comparable to GoDaddy or NameCheap.
- **Register and manage domains:** Allows you to register new domains directly within AWS. Manage existing domains and renew them.
- **Create various record sets on a domain:** A record sets, CNAMEs, MX records, etc., to manage traffic routing.
- **Implement complex traffic flows:** Supports advanced routing configurations like blue/green deployments and failovers.
- **Monitor records via health checks:** Continuously checks the health of endpoints to ensure traffic is routed to healthy instances.
- **Resolve VPC's outside of AWS:** Route 53 can manage DNS queries for resources inside and outside of AWS.

Use Case

- Use Route 53 to point custom domains to AWS resources:
 1. **Incoming Internet Traffic:** Route 53 receives incoming requests and directs them to the appropriate AWS service.
 2. **Elastic Load Balancer (ALB):** Route traffic to your web applications behind an ALB for load balancing.
 3. **EC2 Instance:** Direct traffic to specific instances, useful for tasks like tweaking AMIs.
 4. **API Gateway:** Route traffic to API Gateway which handles API requests.
 5. **CloudFront:** Direct traffic to CloudFront for content delivery, often from S3 static-hosted websites.
 6. **Elastic IP (EIP):** Route traffic to static IPs, such as those hosting specific applications like a Minecraft server.



Route 53 CheatSheet

Hosted Zone: Container for record sets scoped to route traffic for a specific domain or subdomains.

- **Public Hosted Zones:** Route traffic inbound from the Internet.
- **Private Hosted Zones:** Route traffic within an Amazon VPC.

Building SaaS with Subdomains:

- Create a hosted zone domain.
- Create a record set pointing to the subdomain nameservers.
- Create a hosted zone for the subdomain (e.g., app).
- Create a wildcard record set to route traffic back to the root of the subdomain.

Record Sets

- **Record Sets:** Collection of records determining where to send traffic.
- **Batch Changes:** Always changed in batch via API.
- **Record Types Supported:**
 - A, AAAA, CAA, CNAME, DS, MX, NAPTR, NS, PTR, SOA, SPF, SRV, TXT
- **Action Types:**
 - CREATE: Create a new record set.
 - DELETE: Delete an existing record set.
 - UPSERT: Create or update a record set.



Route 53 CheatSheet

Traffic Flow

- **Visual Editor:** Create sophisticated routing configurations using various routing types.
- Supports versioning for updates roll-out or roll-back.
- Various routing types: Weighted, Failover, Geolocation, Latency, Multivalue Answer, Geoproximity.

Routing Policies

- **Simple Routing Policies:** Basic routing policy with one record and multiple IP addresses. Returns all values in random order.
- **Weighted Routing Policies:** Split traffic based on assigned weights. Direct certain percentage to one server, others to different servers.
- **Latency Based Routing:** Direct traffic based on lowest network latency. Requires latency resource record for each region.
- **Failover Routing Policies:** Active/passive setup with primary and secondary sites. Monitors health checks, fails over to secondary if primary is unhealthy.
- **Geolocation Routing Policies:** Direct traffic based on the geographic location of the request.
- **Geoproximity Routing Policies:** Direct traffic based on geographic location with bias value to expand/shrink traffic regions.
- **Multi-Value Answer Policies:** Return multiple values (e.g., IP addresses) for DNS queries. Similar to Simple Routing but includes health checks.



Route 53 CheatSheet

Health Checks:

- Check endpoint health every 30s (can reduce to 10s).
- Can initiate failovers if endpoint is unhealthy.
- Integrate with CloudWatch for alerts.
- Monitor other health checks to create chains of reactions.
- **Capacity:** Up to 50 health checks per AWS account

Route 53 Resolver

- **Amazon Route 53 Resolver:** DNS server for resolving queries between on-premise networks and VPCs.
- **Inbound Endpoints:** Resolve DNS queries to your VPC from on-premises or other VPCs.
- **Outbound Endpoints:** Resolve DNS queries from your VPC to on-premises networks or other VPCs.

Route 53 DNSSEC (Domain Name System Security Extensions)

- Suite of extensions for securing DNS data exchange.
- **Capabilities:** DNSSEC signing validates DNS responses and prevents tampering.
- **Setup:** Create a KSK signing key. Enable DNSSEC for the domain.



Route 53 CheatSheet

Route 53 Zonal Shift

- Capability in Route 53 Application Recovery Controller (ARC).
- Shifts load balancer resources from impaired to healthy Availability Zones.
- **Supported:** Application Load Balancers (ALB) and Network Load Balancers (NLB) without cross-zone load balancing.
- **Not Supported:** ALB as accelerator endpoint in AWS Global Accelerator.
- **Restrictions:** Only for specific load balancers in a single Availability Zone, not multiple zones.

AWS Global Accelerator CheatSheet

Overview

- AWS Global Accelerator: Optimizes path from end user to web servers.
- Edge Locations: Deploys Global Accelerator within Edge Locations.

Accelerator Types

- **Standard:** Automatically routes to the nearest healthy endpoint.
- **Custom Routing:** Routes to specific EC2 instances.

Components

- **Listeners (TCP 3000):** Listens for traffic on a specific port. Sends traffic to an endpoint group.
- **Endpoint Groups:** Collection of endpoints within a specific AWS Region.
 - **Traffic dial:** Adjusts the percentage of traffic.

Endpoint Types

- Network Load Balancer
- Application Load Balancer
- EC2 Instances
- Elastic IP Addresses



EBS CheatSheet

Introduction to EBS

- **IOPS (Input/Output Per Second)**

- Speed of non-contiguous reads and writes on a storage medium.
- High I/O = lots of small, fast reads and writes.

- **Throughput**

- Data transfer rate to and from the storage medium in megabytes per second.

- **Bandwidth**

- Measurement of the total possible speed of data movement along the network.
- Think of Bandwidth as the Pipe and Throughput as the Water.

EBS Volumes

- **General Purpose SSD (gp2)**

- **Use:** General workloads, transactional workloads, virtual desktops, medium-sized databases, low-latency applications, boot volumes, development, and test environments.
- **Durability:** 99.8% - 99.9%.
- **Volume Size:** 1 GiB – 16 TiB.
- **Max IOPS:** 16,000.
- **Max Throughput:** 250 MiB/s.



EBS CheatSheet

- **General Purpose SSD (gp3)**

- **Use:** Similar to gp2 but with up to 20% lower cost per GB.
- **Durability:** 99.8% - 99.9%.
- **Volume Size:** 1 GiB – 16 TiB.
- **Max IOPS:** 16,000.
- **Max Throughput:** 1,000 MiB/s.

- **Provisioned IOPS SSD (io1)**

- **Use:** Sustained IOPS performance or more than 16,000 IOPS, I/O-intensive database workloads.
- **Durability:** 99.8% - 99.9%.
- **Volume Size:** 4 GiB – 16 TiB.
- **Max IOPS:** 64,000.
- **Max Throughput:** 1,000 MiB/s.

- **Provisioned IOPS SSD (io2)**

- **Use:** More durable than io1, sustained IOPS performance.
- **Durability:** 99.999%.
- **Volume Size:** 4 GiB – 64 TiB.
- **Max IOPS:** 256,000.
- **Max Throughput:** 4,000 MiB/s.



EBS CheatSheet

- **io2 Block Express**
 - **Use:** Higher throughput and IOPS, larger storage capacity.
 - **Durability:** 99.999%.
 - **Volume Size:** 4 GiB – 64 TiB.
 - **Max IOPS:** 256,000.
 - **Max Throughput:** 4,000 MiB/s.
- **Cold HDD (sc1)**
 - **Use:** Throughput-oriented storage for infrequently accessed data.
 - **Durability:** 99.8% - 99.9%.
 - **Volume Size:** 125 GiB – 16 TiB.
 - **Max IOPS:** 250.
 - **Max Throughput:** 250 MiB/s.



EBS CheatSheet

- **Throughput Optimized HDD (st1)**
 - **Use:** Big data, data warehouses, log processing.
 - **Durability:** 99.8% - 99.9%.
 - **Volume Size:** 125 GiB – 16 TiB.
 - **Max IOPS:** 500.
 - **Max Throughput:** 500 MiB/s.
- **Magnetic (standard)**
 - **Use:** Workloads where data is infrequently accessed.
 - **Durability:** N/A.
 - **Volume Size:** 1 GiB – 1 TiB.
 - **Max IOPS:** 40-200.
 - **Max Throughput:** 40-90 MiB/s.



EBS CheatSheet

EBS - Moving Volumes

- **To move EBS volumes:**
 - **From one Availability Zone (AZ) to another:**
 1. Take a Snapshot of the volume.
 2. Create an AMI from the Snapshot.
 3. Launch a new EC2 instance in the desired AZ.
 - **From one Region to another:**
 1. Take a Snapshot of the volume.
 2. Create an AMI from the Snapshot.
 3. Copy the AMI to another region.
 4. Launch a new EC2 instance from the copied AMI.

EBS - Encrypted Root Volume

- **Encryption:**
 - EBS Root Volumes are not encrypted by default.
 - Encryption must be turned on at the time of creating an EBS volume or an EC2 instance



EBS CheatSheet

Steps to encrypt an existing volume:

1. Take a Snapshot of the unencrypted volume.
2. Create a copy of the Snapshot and select the Encryption option.
3. Create a new AMI from the encrypted Snapshot.
4. Launch a new EC2 instance from the created AMI.



EBS CheatSheet

EBS Volumes vs Instance Store Volumes

- **EBS Volumes:**

- Durable, block-level storage device that can attach to a single EC2 instance.
- Created from an EBS Snapshot.
- Can start and stop instances, with data persisting if you reboot your system.
- Ideal for when you want data to persist (most use cases).

- **Instance Store Volumes (Ephemeral):**

- Temporary storage type located on disks physically attached to a host machine.
- Created from a template stored on the host machine.
- Cannot stop instances using Instance Store Volumes; only terminate.
- Data is lost if the host fails or the instance is terminated.
- Ideal for temporary backup and storing application cache, logs, or other random data.



EBS CheatSheet

Storage Volumes – HDD

- **Hard Disk Drive (HDD)**

- Uses rotating platters, an actuator arm, and a magnetic head.
- Good for continuous data writing but not for small reads/writes.
- RPMs (Revolutions Per Minute):
 - 7200 RPM: Standard for desktops, good balance of performance and cost.
 - 5400 RPM: Used in laptops and applications with low power consumption.
 - 10000 RPM: High-performance, less common due to SSD rise.

- **HDD RAID Configurations**

- RAID 0 (Striping): No redundancy, data split across disks, minimum 2 disks.
- RAID 1 (Mirroring): Data duplicated, high redundancy, minimum 2 disks.
- RAID 5 (Striping with Parity): Combines striping and parity, minimum 3 disks.
- RAID 6 (Striping with Double Parity): Similar to RAID 5 with double parity, minimum 4 disks.
- RAID 10 (1+0): Combination of RAID 1 and RAID 0, minimum 4 disks.



EBS CheatSheet

Storage Volumes – SSD

- **Solid State Drive (SSD)**

- Uses integrated circuit assemblies, typically using flash memory.
- Resistant to physical shock, quick access time, no moving parts.

- **Types of SSDs**

- SATA SSDs: Most common, good performance, limited by SATA interface.
- NVMe SSDs: Higher performance, uses PCIe interface, available in M.2 and PCIe card forms.
- M.2 SSDs: Compact, suitable for laptops, can use SATA or NVMe interfaces.
- U.2 SSDs: Similar to M.2 but designed for 2.5-inch drive bays, used in enterprise environments.
- Portable SSDs: External drives for easy portability, connected via USB or Thunderbolt.
- PCIe SSDs: Add-on cards for high performance, fits into motherboard PCIe slots.

- **PCIe Slot Generations and Bandwidth**

- PCIe 2.0: 2007, 500 MB/s per lane, 8 GB/s x16 slot.
- PCIe 3.0: 2010, 1 GB/s per lane, 16 GB/s x16 slot.
- PCIe 4.0: 2017, 2 GB/s per lane, 32 GB/s x16 slot.
- PCIe 5.0: 2019, 4 GB/s per lane, 64 GB/s x16 slot.
- PCIe 6.0: 2022, 8 GB/s per lane, 128 GB/s x16 slot.



EBS CheatSheet

Storage Volumes - Magnetic Tape

- A large reel of magnetic tape used for data storage.
- Medium and large-sized data centers use both tape and disk formats.
- Typically comes in the form of a cassette.
- **Advantages:** Durable for decades (up to 30 years). Cheap to produce.



EFS CheatSheet

Elastic File System (EFS): File storage service for EC2 instances.

Features:

- Storage capacity grows (up to petabytes) and shrinks automatically based on data stored.
- Multiple EC2 instances in the same VPC can mount a single EFS Volume.
- EC2 instances install the NFSv4.1 client to mount the EFS volume.
- EFS uses the Network File System version 4 (NFSv4) protocol.
- EFS creates multiple mount targets in all your VPC subnets.

Amazon EFS Client

amazon-efs-utils: Open-source tools for EFS, includes mount helper.

Features:

- Enables Amazon CloudWatch monitoring.
- Mounting on supported EC2 instances (Linux/Mac) with IAM authorization and access points.
- Auto-mounting on EC2 reboot and instance creation.
- On-premise Linux client support.

Limitations: Does not support EC2 Windows instances; previously used standard Linux NFS client.



EFS CheatSheet

Mounting EFS

- **Mount helper** defines a new network file system type, **efs**, compatible with the standard mount command in Linux.
- **Automating mount:** Use `/etc/fstab` configuration with `netdev` option.
- Mount using:
 - File system DNS name.
 - File system ID.
 - Mount target IP address.

EFS Mount Options

- `nfsvers=4.1`: EC2 Linux instances.
- `nfsvers=4.0`: EC2 Mac (macOS Big Sur, Monterey, Ventura).
- `rsz=1048576`, `wsz=1048576`: Max bytes for READ/WRITE to avoid performance issues.
- `hard`: Indefinite retries for data integrity.
- `timeo=600`: 60-second NFS request timeout.
- `retrans=2`: Number of retries before recovery.
- `noresvport`: Uses non-privileged TCP source port after reconnection.
- `mountport=2049`: EC2 Mac (macOS Big Sur, Monterey, Ventura).

Amazon FSx CheatSheet

Amazon FSx

- High-performance file systems in the cloud supporting various protocols:
 - **NetApp ONTAP**: Enterprise storage for petabytes of data
 - **OpenZFS**: Open-source storage, developed by Sun Microsystems
 - **Windows File Server (WFS)**: Native Windows file storage with SSD/HDD support, integrates with Active Directory
 - **Lustre**: Open-source, optimized for parallel computing

Amazon FSx for Windows File Server (WFS)

- **Features**: Windows compatibility, sub-ms latencies, enterprise performance
- **Use Cases**: Business apps, home directories, web/content management, data analytics, software builds, media processing
- **Requirements**: EC2/Workspace Instance, AppStream 2.0, VMWare Cloud on AWS

Amazon File Cache

- High-speed cache for datasets from on-premises/AWS/S3, sub-ms latencies, high throughput
- **Accessible**: EC2, ECS, EKS
- **Compatible AMIs**: Amazon Linux, RHEL, CentOS, Rocky Linux, Ubuntu
- **Integrations**: AWS Batch, Thinkbox Deadline for creative rendering workloads



Snow Family CheatSheet

Purpose: Physical data transfer to/from the cloud when internet/private connections are slow, difficult, or costly.

Snowcone

- **Storage:** 8 TB HDD / 14 TB SSD
- **Ports:** 2 RJ45 (1/10 Gb), USB-C, USB-C PD
- **Features:** E Ink shipping label; Size: 227 x 148.6 x 82.65mm; Weight: 4.5 lbs (2.1 kg); 256-bit encryption, Not HIPAA compliant

Snowball Edge

- **Types:**
 - **Compute Optimized:** 28 TB NVMe SSD
 - **Storage Optimized:** 80 TB HDD / 210 TB NVMe
- **Features:** LCD display, Cluster (3 to 16 nodes); Data Transfer: NFSv3, v4, v4.1, S3 over HTTP/HTTPS; Size: 548 x 320 x 501mmWeight: 49.7 lbs (22.3 kg); HIPAA compliant

Snowmobile

Storage: 100 PB

Form: 45-foot shipping container on a semi-trailer truck

Security: GPS, alarm, 24/7 video, optional escort vehicle

Process: AWS assists with network connection and data transfer



EC2 Auto Scaling Groups CheatSheet

- An ASG is a collection of EC2 instances grouped for scaling and management
- Scaling Out is when adding servers
- Scaling In is when you remove servers
- Scaling Up is when you increase the size of an instance (eg. updating Launch Templates with a larger size)
- Size of an ASG is based on a **Min**, **Max**, and **Desired Capacity**
- **Target Scaling policy** scales based on when a target value for a metric is breached (eg. Average CPU Utilization exceeds 75%)
- **Simple Scaling** policy triggers a scaling when an alarm is breached
- **Scaling Policy with Steps** is the new version of Simple Scaling policy and allows you to create steps based on escalation alarm values.
- Desired Capacity is how many EC2 instances you want to ideally run
- An ASG will always launch instances to meet minimum capacity
- Health checks determine the current state of an instance in the ASG
- Health checks can be run against either an ELB or the EC2 instances
- When an Auto Scaling launches a new instance, it uses a Launch Template which holds the configuration values for that new instance eg. AMI, InstanceType, Role
- Launch Template can be updated using the AWS Management Console, AWS CLI, or SDKs



Service Catalog *CheatSheet*

- **AWS Service Catalog** enables organizations to **create and manage catalogs of products** that are **approved for use** on AWS to achieve consistent governance and meet compliance requirements.
- There are two types of Service Catalog users:
 1. **Catalog Administrator**: They managed the catalog
 2. **End Users**: They use the catalog
- A **product** is a **CloudFormation template** that defines the resources that will be launched
 - You can create or associate an **AWS Budget** with a product
 - Once a product is created, it cannot be edited, only **deleted**.
 - Also, a product must be removed from the portfolio and not provisioned by a user in order to delete.
 - In order for products to be made visible to users, it needs to be added to a portfolio.
- A **Portfolio** is a **collection of products** with the following options:
 - **Constraints** can restrict how products are used.
 - **Permissions** determine who can see and launch products by associating either **Groups, roles, or users**
 - You cannot restrict certain products to certain users in a portfolio, you have to just make additional portfolios
- When a Product is launched, it's called a **Provisioned Product**
- You can attach **Service Actions** (Which are just SSM Documents) to a product so end-users can perform governed actions on the product.



CloudWatch *CheatSheet*

- The Pillars of Observability are **Metrics**, **Logs**, and **Traces**. You need to use all three together to obtain **Observability**
- AWS CloudWatch is a collection of monitoring services:
 - **CloudWatch Logs** — centralization log management service. Logs from AWS Services, Application Logs running on your EC2s
 - **CloudWatch Log Insights** — Robust way to filter your logs. It auto-detects log patterns, creates dynamic fields, and composes queries.
 - **CloudWatch Metrics** — Aggregates data points from AWS Service to produce a metric. (eg AvgCPUUtilization)
 - **CloudWatch Alarms** — Define a threshold up on a metric and reacts by triggering action such as an EC2, Auto Scaling, or SNS action
 - **CloudWatch Events (EventBridge)** — A serverless account-wide event bus. Create event-driven loosely coupled apps with AWS Services
 - **CloudWatch Dashboards** — Create dashboards filled with graphs powered by CloudWatch metrics
 - **CloudWatch Container Insights** — collect, aggregate, and summarize metrics and logs from your containerized apps and microservices
 - **CloudWatch Contributor Insights** — view the top contributors impacting the performance of your systems and applications in real-time
 - **CloudWatch Synthetics** — Test your web apps to see if they're broken. Uses Puppeteer underneath.
 - **CloudWatch Service Lens** — "Observability" for distributed or serverless apps by pulling X-Ray Traces, CloudWatch Logs, and Metrics



CloudWatch Logs *CheatSheet*

CloudWatch Logs is used to **monitor**, **store**, and **access** your log files

- Export logs to S3 to analyze with Athena
- Stream logs to ElasticSearch Service to run a full-text search
- Stream CloudTrail Events to CloudWatch Logs to allow you to react to CloudTrail Events
- Encrypted by default, You can apply your own KMS Key
- Retains logs indefinitely by default. You can choose a custom retention period between 1 Day and 10 years
- Support a simple Filtering Syntax
- **Log Groups** — A container for the collection of log streams. Uses the forward slash naming convention (eg. /my-app/prod/us-east/)
- **Log Streams** — Represent a log, usually created by AWS Services, which can be manually created
- **Log Events** — Represents a single event (a single line from the log file)
- In order to send custom application logs running on your EC2 instance, you need to install the **SSM CloudWatch Agent**
- When the agent is installed, you update the **/etc/awslogs/awslogs.conf** on your EC2 instance to include the logs you want to send to CloudWatch



CloudWatch Logs Insights *CheatSheet*

CloudWatch Logs Insights enables you to **interactively search and analyze your CloudWatch log data**

- CloudWatch Logs Insights supports all kinds of logs
- CloudWatch Logs Insights is generally used from the AWS Console to create ad-hoc queries
- CloudWatch Logs Insights has its own language called CloudWatch Logs Insights **Query Syntax**
- A single request can query up to **50 log groups**.
- Queries **time out after 15 minutes**, if they have not been completed.
- Query results are **available for 7 days**.
- CloudWatch Logs Insights comes with a bunch of sample queries
- CloudWatch Logs Insights analyzes the log events in imported logs to generate **Dynamic Fields**
- CloudWatch Logs Insights always generates out 5 of the following fields:
 - **@message** — the raw unparsed log event.
 - **@timestamp** — the event timestamp contained in the log event's timestamp field.
 - **@ingestionTime** — the time when the log event was received by CloudWatch Logs.
 - **@logStream** — the name of the log stream that the log event was added to.
 - **@log** — is a log group identifier in the form of account-id:log-group-name.
- CloudWatch auto-discovery fields for the following AWS Services
 - Amazon VPC Flow Logs
 - Amazon Route53
 - AWS Lambda
 - AWS CloudTrail
- JSON logs keys will all become discovered fields
- For other logs that are not automatically discovered, you can use the parse command to create ephemeral fields



CloudWatch Metrics *CheatSheet*

CloudWatch Metrics represents a **time-ordered set of data points**. It's a **variable** that is **monitored over time**.

- Metrics are data about the performance of your systems.
- CloudWatch comes with many **predefined** metrics that are generally name-spaced by AWS Service.
- Very common predefined metrics at the EC2 Per-Instance Metrics: CPUUtilization, DiskReadOps, DiskWriteOps, DiskReadBytes, DiskWriteBytes, NetworkIn, NetworkOut, NetworkPacketsIn, NetworkPacketsOut
- Metric data is kept for 15 months, enabling you to view both up-to-the-minute data and historical data.
- **Metric math** enables you to **query multiple CloudWatch metrics** and **use math expressions** to create new time series
- You can publish your own metrics using the AWS CLI or AWS SDK (eg. `aws cloudwatch put-metric-data`)
- You can define custom metrics in two different resolutions:
 - standard resolution (1 minute)
 - high resolution (> 1 minute to 1 second)
- With High-Resolution, you can track in intervals of: 1 second, 5 seconds, 10 seconds, 30 seconds, multiple of 60 seconds.
- Data availability (when you can see the data) varies based on the service
 - **EC2 Basic Monitoring (5 minutes)**
 - EC2 Detailed Monitoring (1 minute)
 - Other Services generally 1 minute but could also be 3 or 5 minutes
- You can turn on EC2 Detailed Monitoring for a price
- Not all EC2 metrics are tracked by default. You have to install the CloudWatch Agent to collect
 - **Memory utilization** (Am I running out of memory?)
 - Disk Swap utilization
 - **Disk Space utilization** (Am I running out of disk space?)
 - Page file utilization



CloudWatch Alarms *CheatSheet*

A **CloudWatch Alarm** monitors a **CloudWatch Metric** based on **a defined threshold**.
When an alarm **breaches** (goes outside the defined threshold), then it changes **state**.

Metric Alarm States

- **OK** The metric or expression is **within** the defined threshold
- **ALARM** The metric or expression is **outside** of the defined threshold
- **INSUFFICIENT_DATA**
 - The alarm has **just started**
 - the metric is **not available**
 - **Not enough data** is available

When it changes state, we can define what **action it should trigger**:

- Notification
- Auto Scaling Group
- EC2 Action
- You can define Conditions of either a **Static** or **Anomaly Detection**
 - **Static** set a static value as the threshold (eg. 100 USD)
 - **Anomaly Detection** sets a band around the data points, helps prevent false positives, and is more flexible if you have seasonal data
- **Composite Alarms** — Allows you to watch multiple alarms and require both to trigger before resulting in an alarm action
 - The alarms being watched must have no actions sets
 - You can only trigger an SNS as the action (so no EC2 or ASG actions)



Kinesis CheatSheet

- **Amazon Kinesis** is the AWS solution for **collecting, processing, and analyzing streaming data** in the cloud. When you need **"real-time"**, *think* Kinesis.
- **Kinesis Data Streams**
 - Per running shard, data can persist within the stream. Data is in order, and every consumer keeps its position. Consumers have to be manually added (coded).
 - Data persists for **24 hours (default)**, up to **8760 hours (365 days)**
 - Inspect data records directly from the AWS Management Console without a dedicated consumer app.
- **Kinesis Firehose**
 - Pay for only the data ingested. Data **immediately disappear** once processed.
 - Consumer of choice is from a predefined set of services: S3, Redshift, OpenSearch Service, Splunk, Datadog, MongoDB, New Relic
 - Supports data transformation before delivering to destinations.
- **Kinesis Data Analytics** (Managed Service for Apache Flink):
 - Perform queries in real-time. Needs Kinesis Data Streams/Firehose as input and output.
 - Supports exactly-once processing semantics and improved connectors.
 - Includes support for Apache Flink version 1.15.



Kinesis CheatSheet

- **Kinesis Video Streams:**
 - Securely ingests and stores video and audio encoded data to consumers such as SageMaker, Rekognition, or other services to apply Machine Learning and video processing.
 - Edge recording and scheduled cloud streaming capabilities (preview).
- **KPL (Kinesis Producer Library):**
 - A Java library to write data to a stream.
 - AWS SDK can be used to write data to streams, but KPL is more efficient.



ElastiCache CheatSheet

- **ElastiCache** is a managed **in-memory** caching service
- ElastiCache can launch either **Memcached** or **Redis**
- **Memcached** is a simple key / value store preferred for caching HTML fragments and is arguably faster than Redis
- **Redis** has richer data types and operations. Great for leaderboard, geospatial data, or keeping track of unread notifications.
- Redis supports replication and clustering for higher availability and scalability.
- Redis offers data persistence options like RDB snapshots and AOF logs.
- A cache is a **temporary storage** area.
- Most frequent identical queries are stored in the cache
- Resources only **within the same VPC** may connect to ElastiCache to ensure low latencies.
- **ElastiCache supports encryption** in transit and at rest, and Redis Auth for securing clusters.
- **Redis supports Lua scripting** for complex operations and Redis Streams for handling message queues.



CloudTrail CheatSheet

- CloudTrail logs calls between AWS services
- **governance, compliance, operational auditing**, and **risk auditing** are keywords relating to CloudTrail
- When you need to know **who to blame**, *think* CloudTrail
- CloudTrail, by default, logs event data for the past 90s days via **Event History**
- To track beyond 90 days, you need to create a **Trail**
- To ensure logs have not been tampered with, you need to turn on **Log File Validation** option
- CloudTrail logs can be encrypted using **KMS (Key Management Service)**
- CloudTrail can be set to log across all AWS accounts in an Organization and all regions in an account.
- CloudTrail logs can be streamed to CloudWatch logs
- Trails are outputted to an S3 bucket that you specify
- CloudTrail logs three kinds of events: **Management events**, **Data events**, and **Insights events**
- **Management events** log management operations (eg. AttachRolePolicy)
- **Data Events** log data operations for resources (S3, Lambda) (eg. GetObject, DeleteObject, and PutObject)
- Data Events are **disabled** by default when creating a Trail.
- Trail which are logged within S3 can be analyzed using Athena



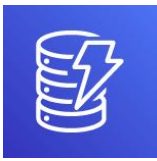
API Gateway CheatSheet

- API Gateway is a solution for creating secure APIs in your cloud environment at any scale.
- Create APIs that act as a front door for applications to access data, business logic, or functionality from back-end services.
- API Gateway throttles API endpoints at **10,000** requests per second (can be increased via service request through AWS support)
- **Stages** allow you to have multiple published versions of your API (eg. prod, staging, QA)
- Each Stage has an **Invoke URL**, which is the endpoint you use to interact with your API
- You can use a custom domain for your Invoke URL (eg. api.exampro.co)
- You need to publish your API via Deploy API. You choose which Stage you want to publish your API
- Resources are your URLs (eg. /projects)
- Resources can have child resources (eg. /projects/**-id-**/**edit**)
- You defined multiple Methods on your Resources (eg. GET, POST, DELETE)
- CORS issues are common with API Gateway, CORS can be enabled on all or individual endpoints
- Caching improves latency and reduces the number of calls made to your endpoint
- Same Origin Policies help to prevent XSS attacks
- Same Origin Policies ignore tools like Postman or curl
- CORS is always enforced by the client.
- You can require Authorization to your API via AWS Cognito or a custom Lambda.



Redshift CheatSheet

- Data can be loaded from S3, EMR, DynamoDB, or multiple data sources on remote hosts.
- Redshift is a Columnar Store database that can SQL-like queries and is an OLAP.
- Redshift can handle petabytes worth of data. Redshift is for Data Warehousing
- Redshift's most common use case is Business Intelligence
- Redshift supports Multi-AZ deployments for higher availability
- Redshift can run via a single node or multi-node (clusters)
- Node sizes vary, with options available beyond 160 GB, tailored to different storage and performance needs
- A multi-node is comprised of a leader node and multiple compute nodes
- You are billed per hour for each node (excluding the leader node in multi-node)
- You are not billed for the leader node
- You can have up to 128 compute nodes
- Redshift has two kinds of Node Types: **Dense Compute** and **Dense Storage**
- Redshift ensures data durability by storing original data on compute nodes, automated backups on S3
- Similar data is stored on disk sequentially for faster reads
- Redshift database can be encrypted via KMS or CloudHSM
- Backup Retention is 1 day by default and can be increased to a maximum of 35 days
- Redshift can asynchronously back up your snapshot to Another Region delivered to S3
- Redshift uses Massively Parallel Processing (MPP) to distribute queries and data across all loads
- In the case of an empty table, when importing, Redshift will sample data to create a schema.



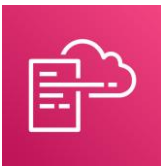
DynamoDB CheatSheet

- **DynamoDB** is a fully managed **NoSQL** key/value and document database.
- Applications that contain large amounts of data but require predictable read and write performance while scaling is a good fit for DynamoDB
- DynamoDB scales with whatever **read and write capacity you specify** per second.
- DynamoDB can be set to have **Eventually Consistent Reads (default)** and **Strongly Consistent Reads**
- **Eventually consistent reads** data is returned immediately, but data can be inconsistent. Copies of data will be generally consistent in 1 second.
- **Strongly Consistent Reads** will wait until data is inconsistent. Data will never be inconsistent, but latency will be higher. Copies of data will be consistent with a guarantee of 1 second.
- DynamoDB stores 3 copies of data on SSD drives across 3 regions.



Cognito CheatSheet

- Cognito is a decentralized managed authentication system. When you need to add authentication to your mobile and desktop app easily, *think* Cognito
- **User Pools** user directory allows users to authenticate using OAuth to IdP, such as Facebook, Google, and Amazon to connect to web applications. Cognito User Pool is in itself an IdP
- User Pools use **JWTs** to persist authentication
- **Identity Pools** provide **temporary AWS credentials** to access services eg. S3, DynamoDB
- **Cognito Sync** can sync **user data** and **preferences** across devices with one line of code (powered by SNS)
- **Web Identity Federation** exchange identity and security information between an identity provider (IdP) and an application
- **Identity Provider (IdP)** a trusted provider of your user identity that lets you use authenticate to access other services. eg. Facebook, Twitter, Google, Amazon
- **OIDC** is a type of Identity Provider that uses OAuth
- **SAML** is a type of Identity Provider that is used for Single Sign-On



CloudFormation Cheat Sheet

- When being asked to **automate** the provisioning of resources, *think* CloudFormation
- When Infrastructure as Code (IaC) is mentioned, *think* CloudFormation
- CloudFormation can be written in either JSON or YAML
- When CloudFormation encounters an error, it will rollback with **ROLLBACK_IN_PROGRESS**
- CloudFormation templates larger than 51,200 bytes (0.05 MB) are too large to upload directly and must be imported into CloudFormation via an S3 bucket.
- **NestedStacks** helps you break up your CloudFormation template into smaller reusable templates that can be composed into larger templates
- **At least one resource** under resources: must be defined for a CloudFormation template **to be valid**
- CloudFormation **Template Sections**
 - **MetaData** extra information about your template
 - **Description** a description of what the template is supposed to do
 - **Parameters** is how you get user inputs into templates
 - **Transforms** Applies macros (like applying a mod that changes the anatomy to be custom)
 - **Outputs** are values you can use to import into other stacks
 - **Mappings** map keys to values, just like a lookup table
 - **Resources** define the resources you want to provision, **at least one resource is required**
 - **Conditions** are whether resources are created or properties are assigned



Elastic Beanstalk CheatSheet

- **Elastic Beanstalk** handles the deployment, from capacity provisioning, load balancing, and auto-scaling to application health monitoring
- When you want to run a web application, but you don't want to have to think about the underlying infrastructure.
- It costs nothing to use Elastic Beanstalk (only the resources it provisions eg. RDS, ELB, EC2)
- Recommended for test or development apps. Not recommended for production use
- You can choose from the following pre-configured platforms: Java, .NET, PHP, Node.js, Python, Ruby, Go, and Docker
- You can run dockerized environments on Elastic Beanstalk.



Storage Gateway CheatSheet

- **Storage Gateway** connects on-premise storage to cloud storage (hybrid storage solution)
- There are four types of Gateways: File Gateway, Volume Gateway, Tape Gateway, and Amazon FSx File Gateway.
- **File Gateway**
 - Lets S3 act as a local file system using NFS or SMB, extends your local hard drive to S3.
 - Supports creating multiple file shares for a single S3 bucket.
 - Local cache size increased up to 64 TiB, providing low-latency access to larger working datasets.
 - Supports S3 Standard, S3 Intelligent-Tiering, S3 Standard-IA, and S3 One Zone-IA.
 - Use cases: migrating on-premises file data to Amazon S3, backing up on-premises file data as objects in Amazon S3, hybrid cloud workflows.
- **Amazon FSx File Gateway**
 - Provides low-latency, on-premises access to fully managed file shares for team file shares, home directories, and file-based business applications.
- **Tape Gateway**
 - Backs up virtual tapes to S3 Glacier for long archive storage.
 - Supports seamless archiving to S3 Glacier and S3 Glacier Deep Archive for cost savings.



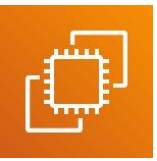
Storage Gateway CheatSheet

- **Volume Gateway**

- Used for backups and has two types: Stored and Cached.
- Stored Volume Gateway continuously backs up local storage to S3 as EBS Snapshots. Primary Data on-premise.
- Cached Volume Gateway caches frequently used blocks on-premise. Primary Data is stored on S3.
- Integrates with AWS Backup for centralized and automated backup management.
- Stored Volumes: 1GB to 16TB.
- Cached Volumes: 1GB to 32GB.

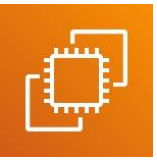
- **Additional Features and Improvements:**

- Encryption: Supports AES-128 or AES-256 encryption.
- Log Management: Enhanced system log management to handle low disk space scenarios.
- CloudWatch Integration: File Gateway can be configured with Amazon CloudWatch Log Groups for health and error notifications.



EC2 Pricing – CheatSheet

- EC2 has for 4 pricing models **On-Demand**, **Spot**, **Reserved Instances (RI)**, and **Dedicated**
- **On-Demand** (least commitment)
 - low cost and flexible
 - only pay per hour
 - **Use case:** short-term, spiky, unpredictable workloads, first-time apps
 - Ideal when your workload cannot be interrupted
- **Reserved Instances** up to 75% off (Best long-term value)
 - **Use case:** steady state or predictable usage
 - Can resell unused reserved instances (Reserved Instance Marketplace)
 - Reduced Pricing is based on **Term** x **Class Offering** x **Payment Option**
 - **Payment Terms:** 1-year or 3-year
 - **Payment Options:** All Upfront, Partial Upfront, and No Upfront
 - **Class Offerings**
 - **Standard** Up to 75% reduced pricing compared to on-demand. Cannot change RI Attributes.
 - **Convertible** Up to 54% reduced pricing compared to on-demand. Allows you to change RI Attributes if greater or equal in value.
 - **Scheduled** You reserve instances for specific time periods eg. once a week for a few hours. Savings vary



EC2 Pricing – CheatSheet

- **Spot Pricing** up to 90% off (Biggest Savings)
 - request spare computing capacity
 - flexible start and end times
 - **Use case:** Can handle interruptions (server randomly stopping and starting)
 - **Use case:** For non-critical background jobs
 - Instances can be terminated by AWS **at anytime**
 - If **AWS terminates** your instance, **you don't get charged** for a partial hour of usage.
 - If **you terminate** your instance, **you will still be charged** for any hour it ran.
- **Dedicated Hosting** (Most Expensive)
 - Dedicated servers
 - Can be on-demand or reserved (up to 70% off)
 - **Use case:** When you need a guarantee of isolated hardware (enterprise requirements)